Xây dựng hệ thống giám sát và phát hiện tấn công web dựa trên phân tích access log

Lê Quang Huy[1]

Nhóm G21, lớp 26.1A01, học phần Chính sách và Quản trị An ninh Mạng (IT6027)
Mã sinh viên: 20261106M
huy.lq261106m@sis.hust.edu.vn

Tóm tắt nội dung Báo cáo trình bày việc thiết kế, hiện thực và đánh giá một hệ thống giám sát tấn công web từ access log cho phòng thực hành SOC. Khác với Web Application Firewall đặt inline, hệ thống không chặn yêu cầu mà thu thập log JSON của Nginx qua Filebeat và Kafka, sau đó chuẩn hóa, áp dụng 14 chữ ký biểu thức chính quy, chấm điểm rủi ro, sinh cảnh báo và tương quan sự cố trong Spring Boot. PostgreSQL lưu bằng chứng, Redis hỗ trợ bộ nhớ đệm, giao diện React nhận cập nhật qua Server-Sent Events; mô hình ngôn ngữ Ollama chỉ giải thích sự cố bất đồng bộ và không tham gia quyết định phát hiện. Báo cáo xây dựng mô hình đe dọa cho SQL injection, Cross-Site Scripting và Path Traversal, đồng thời ánh xạ kiểm soát vận hành với NIST CSF 2.0 và ISO/IEC 27001. Đánh giá được tách thành các tầng không trộn mẫu số: bộ luật offline, thí nghiệm đầu cuối trên OWASP Juice Shop và OWASP Benchmark. Trên 31 probe cần cảnh báo, hệ thống đạt recall 0,774 và không tạo dương tính giả trên 56 yêu cầu gắn nhãn sạch. Thời gian từ yêu cầu đến cảnh báo là 11,5–15,1 giây, chưa đạt mục tiêu dưới 5 giây. Kết quả cho thấy giải pháp phù hợp cho minh họa detection engineering và quản trị sự cố, nhưng chưa thay thế WAF inline và còn bị giới hạn bởi dữ liệu chỉ có trong access log.

Keywords: Web security · Access-log analysis · Kafka · Incident management · NIST CSF · OWASP

1 Đặt vấn đề và mục tiêu

1.1 Bối cảnh và tính cấp thiết

Ứng dụng web tiếp tục là bề mặt tấn công quan trọng; các nhóm rủi ro như Injection và sai sót kiểm soát đầu vào xuất hiện thường xuyên trong hướng dẫn OWASP [1]. Tại một SOC, lưu lượng lớn làm cho việc đọc log thủ công khó mở rộng: cùng một chiến dịch có thể tạo nhiều sự kiện, trong khi một mẫu bất thường đơn lẻ chưa đủ để kết luận sự cố. Nếu thiếu cơ chế chuẩn hóa, tương quan và lưu bằng chứng, tổ chức có nguy cơ phát hiện chậm, bỏ sót chuỗi tấn công, điều tra không nhất quán và không chứng minh được trách nhiệm xử lý.

Đề tài giải quyết bài toán hẹp nhưng kiểm chứng được: phát hiện dấu hiệu SQL injection (SQLi), Cross-Site Scripting (XSS) và Path Traversal trong access log của Nginx; chuyển các lần khớp thành điểm rủi ro, cảnh báo và sự cố có vòng đời. Hệ thống là *out-of-band log analyzer*, không phải WAF inline. Vì quyết định được đưa ra sau khi Nginx đã chuyển tiếp yêu cầu, sản phẩm hỗ trợ *Detect–Respond*, không cam kết *Prevent*.

1.2 Mục tiêu và tiêu chí thành công

1. Xây dựng pipeline tái lập được từ Juice Shop tới Nginx, Filebeat, Kafka, Spring Boot, PostgreSQL và giao diện SOC.
2. Phát hiện có thể giải thích: mỗi hit chỉ rõ luật, trường đích, bằng chứng và trọng số; không dùng LLM trên đường xử lý quyết định.
3. Giảm nhiễu bằng ngưỡng cảnh báo và chính sách gộp Alert thành Incident theo ứng dụng và địa chỉ IP.
4. Áp dụng JWT/RBAC cho ba vai trò, cung cấp luồng triage `OPEN–ACK–CLOSED`, cập nhật thời gian thực và giải thích tùy chọn.
5. Đánh giá riêng biệt chất lượng luật, hiệu quả đầu cuối, khả năng khái quát và độ trễ; công bố cả kết quả không đạt.

2 Nghiên cứu liên quan

2.1 Các hướng tiếp cận

CSIC 2010 và HttpParamsDataset cung cấp request hoặc giá trị tham số bình thường/bất thường, thường được dùng để huấn luyện bộ phân loại ML [5,6]. Các mô hình TF–IDF/SVM có thể cho độ chính xác cao trên phép chia tĩnh, nhưng nhãn dự đoán khó chuyển trực tiếp thành bằng chứng luật và quy trình xử lý sự cố.

ModSecurity Core Rule Set đại diện cho hướng WAF inline, thực hiện luật trước ứng dụng [7]. WAFBooster sinh biến thể để mô phỏng WAF hộp đen rồi trích chữ ký [8]; GenXSS dùng LLM tạo payload, gom cụm và sinh SecRule cho XSS [9]. Các hướng này nhắm tới allow/block, cần môi trường và metric khác với bộ phân tích log hậu kiểm. Các stack SIEM dựa trên Beats/Kafka/ELK mạnh về thu thập và tìm kiếm; Kafka cung cấp cơ chế log phân tán, tách producer và consumer [12]. Tuy nhiên một stack thu log không tự giải quyết nguồn gốc luật, ngưỡng cảnh báo hay chính sách Incident.

Bảng 1. Đối chiếu các hướng liên quan và đề tài.

Tiêu chí	ML trên payload	WAF/sinh SecRule	SIEM log pipeline	Giải pháp đề xuất
Vị trí	Request/dataset	Inline trước ứng dụng	Sau nguồn log	Sau Nginx, qua Kafka
Detector	Classifier	CRS/chữ ký	Rule/analytics đa dạng	14 regex Java có evidence
Đầu ra	Nhãn attack	Allow/block	Event/alert	Hit–Alert–Incident
LLM	Có thể là detector	Sinh payload/rule	Trợ lý tùy chọn	Chỉ explain sau promote
Giới hạn chính	Drift, khó giải thích	FP và tài nguyên	Stack nặng, alert fatigue	Không body, không block

2.2 Khoảng trống và đóng góp

Không công trình được khảo sát kết hợp đầy đủ bốn đặc điểm trong cùng lab: (i) phân tích access log hậu kiểm; (ii) luật được khai phá offline, qua compiler, held-out và cổng

FPR; (iii) Alert khác Incident với chính sách gộp định lượng; (iv) LLM chỉ giải thích sau khi detector tất định đã ra quyết định. Đóng góp của đề tài nằm ở sự tích hợp có thể tái lập và khung quản trị, không phải tuyên bố một thuật toán phát hiện hoàn toàn mới. Do metric và vị trí triển khai khác nhau, báo cáo không tuyên bố hệ thống tốt hơn WAFBooster, GenXSS hay CRS.

2.3 So sánh định lượng có kiểm soát

Bảng 2 đặt các con số công khai cạnh kết quả đề tài để cho thấy quy mô và xu hướng, không tạo bảng xếp hạng. Đơn vị mẫu khác nhau: payload, HTTP request, bypass đã chọn hoặc request production; điểm quyết định khác nhau: block inline so với Alert hậu kiểm. Vì vậy chỉ có thể so sánh định tính về trade-off.

Bảng 2. Số liệu từ công trình gốc và giới hạn so sánh.

Nguồn	Quy mô/phạm vi	Kết quả công bố	Quan hệ với đề tài
WAFBooster [8]	8 WAF; SQLi/XSS/command injection; inline	TRR trung bình payload đột biến 21% lên 96%, FRR 0; ModSecurity SQLi 16% lên 96%	TRR cùng hướng với recall, nhưng tập bypass dùng để sinh signature, workload và vị trí block khác
GenXSS [9]	174 XSS đã bypass ModSecurity; 800 benign; 15 SecRule	TP 150, FN 24, recall 86,21%, precision 100%, FP 0	Chỉ XSS và inline; confusion matrix dùng 174 attack dù paper tạo 220 payload hợp lệ
ModSec-Learn [10]	Independent test 5.000 benign + 5.000 SQLi; inline CRS feature	Tại FPR 1%: ModSecurity PL2 TPR 75,45%; Random Forest 99,41%	Metric được khóa FPR tốt, nhưng classifier SQLi-only và mẫu lớn hơn hẳn 11 probe SQLi của đề tài
RuleCache [11]	861.988 request thực trong 6 ngày; throughput inline	Nginx 16,32 kRPS; Nginx+ModSecurity/CRS 1,72 kRPS; cải thiện tới 5,5 lần	Là throughput inline, không so được với 11,5–15,1 giây từ request qua Filebeat/Kafka tới Alert API
Đề tài	31 attack probe + 56 clean request trên Juice; 14 rule, ba category	TP 24, FN 7, FP 0; recall 77,4%; request–Alert 11,5–15,1 giây	Bằng chứng đầu cuối nhỏ, application-specific; không đại diện production WAF

Quy mô corpus cũng cần đọc đúng. CSIC 2010 có 36.000 request normal cho train, 36.000 normal cho test và 25.065 anomalous [5]. HttpParamsDataset có 31.067 giá trị tham số: 19.304 benign và 11.763 anomalous, gồm 10.852 SQLi, 532 XSS, 290 traversal và 89 command injection [6]. Sau normalize/deduplicate, đề tài dùng 12.099 benign value cho FPR từng rule. Đây là provenance của dữ liệu, không phải external baseline độc lập.

3 Kiến thức nền tảng và mô hình đe dọa

3.1 Log, signature và tương quan sự cố

Access log ghi metadata đã quan sát tại reverse proxy: thời gian, host, IP, phương thức, đường dẫn thô, query, user-agent, mã trạng thái và độ trễ upstream. Signature detector đối chiếu chuỗi với pattern đã biết. Ưu điểm là tất định và truy vết được; nhược

điểm là coverage phụ thuộc luật, dễ bỏ sót biến thể hoặc zero-day. Hệ thống vì vậy lưu `DetectionHit` riêng, sau đó mới tính điểm và tương quan theo cửa sổ thời gian.

Với tập hit H_e của event e, điểm được hiện thực như sau:

$$S(e) = \min\left(100, \sum_{h \in H_e} w_h + B_{\text{freq}} + B_{\text{status}}\right).$$

$B_{\text{freq}} \leq 20$ phụ thuộc số hit cùng IP trong 5 phút; $B_{\text{status}} = 5$ khi đã có hit và status thuộc $\{403, 404, 500\}$. Alert được mở khi $S(e)$ không nhỏ hơn `applications.risk_threshold`, mặc định 60.

3.2 Tài sản, biên tin cậy và giả định

Tài sản cần bảo vệ gồm tính toàn vẹn của log/bằng chứng, thông tin xác thực SOC, luật phát hiện, trạng thái Incident và tính sẵn sàng pipeline. Đối thủ bên ngoài điều khiển URL, query, user-agent và tần suất request; người dùng SOC có thể lạm quyền nếu RBAC sai. Biên tin cậy nằm giữa Internet–Nginx, Filebeat–Kafka, backend–cơ sở dữ liệu và backend–Ollama. Lab giả định host Docker thuộc quyền quản trị, đồng hồ các container đủ đồng bộ và probe chỉ nhắm Juice Shop tự lưu trữ.

Bảng 3. Bản chất và tác động của các đe dọa trong phạm vi.

Đe dọa	Cơ chế	Tác động
SQLi	Chèn toán tử, UNION, comment hoặc hàm time-based để thay đổi câu SQL	Rò rỉ/sửa dữ liệu, bypass đăng nhập
XSS	Chèn script/HTML/event handler để thực thi trong trình duyệt nạn nhân	Chiếm phiên, giả mạo nội dung
Path Traversal	Dùng `../`, encode hoặc double-encode để thoát thư mục gốc	Đọc file, lộ secret/cấu hình
Log evasion	Đặt payload trong POST body/header không được ghi hoặc dùng encoding chưa có luật	Tạo false negative có hệ thống
SOC compromise	Đánh cắp JWT, credential yếu hoặc lạm quyền	Sửa luật, đóng sự cố, xem dữ liệu trái phép
Prompt injection	Dữ liệu log cố điều khiển LLM	Làm sai nội dung giải thích, không đổi verdict detector

SQLi, XSS và Path Traversal chỉ bị *phát hiện* khi biểu hiện nằm trong `path`, `query`, `ua` hoặc `raw`. Biện pháp phòng ngừa đúng vẫn phải đặt tại ứng dụng hoặc WAF inline; analyzer là lớp giám sát bù trừ.

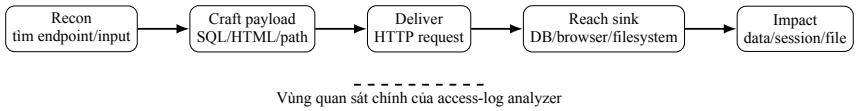
3.3 Vòng đời một cuộc tấn công web trong phạm vi đề tài

Một payload đáng ngờ không tự động đồng nghĩa ứng dụng đã bị khai thác. Cần phân biệt bốn mức: (1) *probe* thăm dò; (2) *attack attempt* có cú pháp tấn công; (3) *exploitation*

Bảng 4. Dấu hiệu quan sát và kiểm soát phòng vệ.

Đe dọa	Dấu hiệu có thể quan sát	Kiểm soát phòng vệ
SQLi	Token SQL trong path/query; chuỗi encode; nhiều request cùng IP	Prepared statement, WAF inline, luật SQLI và scoring
XSS	`script`, `javascript:`, event handler trong URL	Contextual output encoding, CSP, sanitize và luật XSS
Path Traversal	Path thô chứa traversal hoặc dạng encode	Canonicalize, allow-list, quyền file tối thiểu và luật trên `raw`
Log evasion	Không có bằng chứng trong schema hiện tại	Telemetry mở rộng có masking/giới hạn; adversarial test
SOC compromise	Login/API bất thường	BCrypt, JWT, RBAC, secret production và audit log
Prompt injection	Chuỗi lệnh trong field đưa vào prompt	Clip/sanitize, prompt cố định; LLM không được đổi verdict

khi payload thực sự đi vào sink dễ tổn thương; và (4) *impact* như đọc dữ liệu hoặc chiếm phiên. Access log của đề tài quan sát tốt mức (1)–(2), nhưng thường không đủ dữ kiện chứng minh mức (3)–(4). Đây cũng là nguyên nhân nhãn "request có dấu hiệu tấn công" khác nhãn "đoạn mã server có lỗ hổng" trong OWASP Benchmark.



Hình 1. Các giai đoạn từ thăm dò tới tác động và giới hạn quan sát.

Ba nhóm được chọn vì chúng có biểu hiện URL/query đủ rõ để minh họa signature detection và có dữ liệu đánh giá trong repository. CSRF, SSRF, deserialization, command injection, authentication attack và business-logic abuse không có production rule; báo cáo không tuyên bố phát hiện các nhóm đó.

3.4 SQL Injection

Bản chất và cơ chế hoạt động SQL injection xảy ra khi dữ liệu không tin cậy được ghép vào câu lệnh SQL như một phần cú pháp thay vì được truyền dưới dạng tham số [2]. Ví dụ khái niệm:

$$\texttt{SELECT} * \texttt{FROM users WHERE name='}\mathit{input}\texttt{'}$$

Nếu *input* đóng dấu nháy và bổ sung toán tử, cấu trúc parse tree của câu SQL thay đổi. Lỗ hổng nằm ở cách ứng dụng tạo query; chuỗi `OR 1=1` trong log chỉ là dấu hiệu attempt, không chứng minh backend đã ghép chuỗi hoặc database đã thực thi.

Boolean-based. Kẻ tấn công chèn mệnh đề luôn đúng như `OR 1=1` hoặc `OR 'a'='a'` để mở rộng tập kết quả hoặc bypass điều kiện xác thực. Hai rule `SQLI-BOOLEAN-001/002` nhận dạng dạng số và chuỗi, trọng số 40. Một hit đơn thường chưa qua threshold 60; nếu đi kèm UNION hoặc tần suất cao, score mới tăng.

UNION-based. `UNION SELECT` nối kết quả query do attacker điều khiển vào query gốc. Muốn khai thác thành công thường phải khớp số cột và kiểu dữ liệu; error message hoặc thử `ORDER BY` có thể hỗ trợ reconnaissance. Rule `SQLI-UNION-001` có trọng số 55. Probe `sqli-union-bool` đồng thời khớp BOOLEAN và UNION, base score 95, được xếp CRITICAL trước bonus.

Error-based và comment. Attacker cố tạo lỗi để suy ra DBMS/schema, hoặc dùng `--`, `#`, `/*` để vô hiệu phần query phía sau. Hệ thống hiện có `SQLI-COMMENT-001` trọng số 35, nhưng không có rule tổng quát riêng cho mọi hàm error-based. Status 500 chỉ cộng 5 nếu đã có hit; status không tự tạo verdict vì lỗi ứng dụng thông thường cũng có thể trả 500.

Stacked-query. Dấu chấm phẩy kết thúc câu hiện tại rồi mở câu `SELECT`, `UPDATE`, `DROP`, `EXEC` hoặc `WAITFOR`. Khả năng thành công phụ thuộc driver có cho multiple statements hay không. Rule `SQLI-STACKED-001` trọng số 50; probe `sqli-stacked-sleep` ghép stacked và time-delay.

Blind/time-based. Khi response không lộ dữ liệu, attacker mã hóa bit thông tin bằng khác biệt đúng/sai hoặc thời gian. Primitive trong lab gồm `SLEEP`, `pg_sleep`, `BENCHMARK` và `WAITFOR DELAY`; `SQLI-TIME-001` có trọng số 60. Tuy nhiên bốn probe time-based trong artifact có trường hợp FN vì chuỗi runtime không khớp như giả định hoặc event/alert ghép không thành công. Analyzer cũng không so sánh latency response để xác nhận delay đã xảy ra.

Tác động và điều kiện thành công Tác động có thể gồm bypass login, đọc credential, sửa/xóa dữ liệu và, với quyền DB quá lớn, thực thi lệnh hệ điều hành. Mức độ phụ thuộc sink, quyền database account, khả năng chạy nhiều statement, loại DBMS và dữ liệu trả về. Incident phải ghi "dấu hiệu SQLi" thay vì khẳng định "database đã bị xâm nhập" khi chưa có bằng chứng từ ứng dụng hoặc DB.

Dấu hiệu, phòng ngừa và xử lý quản trị

3.5 Cross-Site Scripting

Bản chất và cơ chế hoạt động XSS xảy ra khi dữ liệu attacker kiểm soát đi vào HTML/DOM sink mà không được encode hoặc sanitize theo đúng ngữ cảnh, khiến trình duyệt diễn giải nó như mã trong origin của ứng dụng [3]. Khác SQLi chạy ở server/database, payload XSS thường chỉ gây tác động khi một browser nạn nhân render dữ liệu.

Bảng 5. Kiểm soát nhiều lớp đối với SQL injection.

Lớp	Kiểm soát	Bằng chứng/giám sát
Ứng dụng	Prepared statement/parameterized query; allow-list cho tên cột/sort; không nối chuỗi SQL	Code review, SAST và test integration
Database	Tài khoản least privilege; tách read/write; hạn chế dangerous function; backup	DB audit, query error và thay đổi dữ liệu
Edge	WAF inline/CRS để block sau tuning	WAF audit log, rule id
SOC analyzer	BOOLEAN, UNION, COMMENT, STACKED, TIME; score/correlation	WebEvent, DetectionHit, Alert và Incident
Ứng phó	ACK, xác minh response/DB log, cô lập credential nếu có impact, vá sink rồi regression test	Timeline, owner, evidence và closure note

Reflected XSS. Payload từ URL/query được phản chiếu ngay vào response. Attacker thường gửi link đã craft cho nạn nhân. Probe `xss-script-js` và `xss-svg` mô phỏng tag script/SVG trong query. Access log nhìn thấy attempt, nhưng không nhìn response body nên không biết payload có được phản chiếu hay encode.

Stored XSS. Payload được lưu vào database rồi hiển thị cho nhiều người dùng, nên một request ghi dữ liệu có thể tách xa request kích hoạt. Vì schema log không có POST body và hệ thống không theo dõi data flow tới trang render, stored XSS body-only là false negative theo thiết kế.

DOM-based XSS. JavaScript phía client lấy dữ liệu từ source như URL fragment, `location` hoặc message rồi đưa vào sink như `innerHTML`/`eval`. URL fragment không được gửi lên HTTP server, vì vậy Nginx access log không thể quan sát nhiều DOM-XSS vector.

Các dạng payload trong lab. Bốn rule production bao phủ: tag `script`/`iframe`/`svg` (weight 55), event handler `onerror`/`onload` (45), `javascript:`/`vbscript:` URI (50), và opening tag percent hoặc double encoded (40). Các probe kết hợp nhiều primitive nên thường đạt score trên 60; ví dụ IMG+`onerror`+`javascript:` tạo hai hit. Pattern match không chứng minh JavaScript đã thực thi.

Tác động và kiểm soát Mã chạy trong origin có thể đọc dữ liệu mà JavaScript được phép đọc, thực hiện request với phiên nạn nhân, thay đổi DOM, phishing hoặc phát tán worm. Cookie `HttpOnly` giảm khả năng đọc session cookie nhưng không ngăn attacker gọi API cùng origin. Phòng vệ chính là output encoding theo context (HTML, attribute, JavaScript, URL), template auto-escaping, sanitizer đáng tin cậy khi cho phép HTML, tránh unsafe DOM sink và validate URL scheme. Content Security Policy là lớp giảm thiểu, không thay thế việc encode đúng.

Bảng 6. Khả năng quan sát XSS của hệ thống.

Vector	Access log hiện tại	Kết luận cho SOC
Reflected qua query	Thấy URL-encoded query và rule hit	Có attempt; cần kiểm tra response/sink để kết luận exploit
Stored qua POST body	Không thấy body	Có thể FN; cần app audit/event hoặc body telemetry đã masking
DOM qua query	Có thể thấy query	Không biết DOM sink có thực thi
DOM qua URL fragment	Không được gửi tới server	Ngoài khả năng quan sát của Nginx log
CSP violation	Chưa ingest CSP report	Nguồn telemetry nên bổ sung

3.6 Path Traversal

Bản chất và cơ chế hoạt động Path Traversal xuất hiện khi ứng dụng dùng input để dựng đường dẫn file mà không ràng buộc kết quả cuối trong thư mục cho phép [4]. Chuỗi `../` trên Unix hoặc `..\` trên Windows yêu cầu filesystem lùi một cấp. Nếu code nối `baseDirectory + userInput`, attacker có thể thoát document root và truy cập file ngoài ý định.

Encoded separator và dot. `%2e`, `%2f`, `%5c` biểu diễn dot, slash và backslash. Bypass xảy ra khi proxy/WAF validate chuỗi encoded nhưng framework decode sau đó. Mixed separator giúp payload chạy trên parser khác nhau.

Double encoding. `%252e` decode lần đầu thành `%2e`, lần hai thành dấu chấm. Nếu mỗi layer decode một lần, validation và file access nhìn hai giá trị khác nhau. Các probe `pt-double` và `hard-pt-double` kiểm tra hiện tượng này.

Absolute path, null byte và normalization. Một số stack chấp nhận absolute path hoặc từng bị ảnh hưởng bởi null-byte termination. Đây là kiến thức cần kiểm thử theo ngôn ngữ/framework, nhưng 14 rule hiện tại chỉ bao phủ dot-dot, slash/backslash encoded và double encoding; báo cáo không tuyên bố phát hiện mọi path manipulation.

Tác động, detection và phòng ngừa Tác động điển hình là đọc `/etc/passwd`, file cấu hình, source code, private key hoặc `win.ini`; nếu endpoint cho ghi/upload, traversal có thể ghi đè file hoặc dẫn tới code execution. Migration V3 đổi bốn rule traversal sang target `RAW`, phù hợp việc Nginx ghi raw request path và query trước khi ứng dụng canonicalize. Hệ thống vẫn không biết file có tồn tại hay response đã chứa nội dung.

Phòng ngừa ưu tiên không nhận path trực tiếp: ánh xạ ID sang file server quản lý. Nếu phải nhận path, decode đúng một quy trình xác định, resolve canonical path, rồi kiểm tra nó vẫn nằm dưới canonical base directory; dùng allow-list extension/tên, quyền filesystem tối thiểu và không trả đường dẫn nội bộ trong lỗi. Validation phải diễn ra sau canonicalization phù hợp, tránh kiểm tra một biểu diễn rồi sử dụng biểu diễn khác.

Bảng 7. Kỹ thuật evasion được phản ánh trong probe thực tế.

Kỹ thuật	Ví dụ trong artifact	Rủi ro còn lại
Mixed case/space	`hard-sqli-mixed-case`; dấu cách dạng `+`/`%20`	Runtime không canonicalize giống offline pipeline
Percent encoding	XSS `%3cscript`; traversal `%2e%2e%2f`	Decode khác nhau giữa Nginx/framework
Double encoding	`%253csvg`, `%252e%252e%252f`	Pattern chỉ bao phủ một số depth/primitive
Comment/fragment	quote+`--`; stacked statement	Obfuscation bằng comment nội token có thể miss
Multi-category	`hard-multi`: SQLi+XSS+traversal	Một query có nhiều hit; cần triage theo toàn bộ evidence
Invisible body/header	Payload trong POST body/cookie	Không có trong schema access log, tạo FN có hệ thống
Low-and-slow	Request tách theo thời gian/IP	Cửa sổ 5 phút và key hiện tại có thể không correlate

3.7 Kỹ thuật né tránh và tấn công kết hợp

Việc thêm decode không thể làm tùy tiện: decode quá mức có thể biến dữ liệu benign thành pattern nguy hiểm hoặc tạo khác biệt so với ứng dụng. Detection engineering cần ghi rõ representation được match, giữ raw value làm bằng chứng, và regression cả payload lẫn benign corpus sau mỗi thay đổi.

3.8 Outcome quản trị: từ dấu hiệu tới quyết định xử lý

Đối với môn Chính sách và Quản trị An ninh Mạng, outcome không dừng ở nhận diện cú pháp. Analyst phải xác định tài sản, owner, độ tin cậy bằng chứng, tác động tiềm năng và hành động có thẩm quyền:

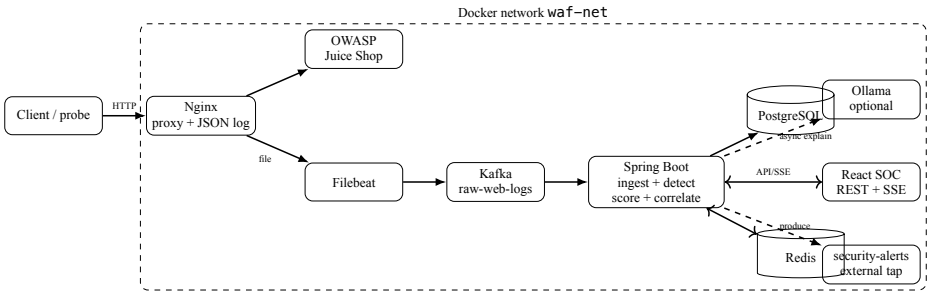
1. **Validate:** kiểm tra event/hit, raw representation, Application, IP, chuỗi request và khả năng false positive.
2. **Scope:** tìm Alert cùng IP/app, endpoint bị nhắm, thời gian và category kết hợp; không suy diễn thành công chỉ từ status code.
3. **Corroborate:** SQLi cần app/DB audit; XSS cần response và browser/CSP report; traversal cần file-access log.
4. **Contain:** nếu xác nhận impact, cô lập endpoint hoặc credential, áp dụng WAF rule tạm có review; analyzer tự nó không block.
5. **Eradicate/Recover:** vá sink gốc, giảm privilege, rotate secret khi cần, restore dữ liệu và chạy regression.
6. **Learn:** cập nhật rule có version/provenance, đo lại held-out/benign/ Juice; không auto-tune từ một Incident đơn.

Như vậy ba nhóm tấn công vừa là đối tượng kỹ thuật của detector vừa là đầu vào cho quy trình quản trị rủi ro, bằng chứng và ứng phó sự cố.

4 Giải pháp đề xuất

4.1 Kiến trúc tổng thể

Hình 2 phản ánh topology trong Docker Compose. Nginx vừa reverse proxy tới Juice Shop vừa ghi access log JSON. Filebeat đóng gói log vào topic `raw-web-logs`. Backend lưu event trước khi gọi pipeline phát hiện. Topic `security-alerts` là kênh xuất cho tích hợp ngoài; giao diện hiện tại nhận SSE trực tiếp từ backend, không consume topic này.



Hình 2. Kiến trúc triển khai và các luồng dữ liệu chính.

4.2 Luồng ingest và phát hiện

`RawWebLogListener` nhận Kafka record và tạo `rawRef` từ topic–partition–offset để chống ghi trùng. `NginxAccessLogParser` đọc JSON trực tiếp hoặc trường `message` trong envelope của Filebeat. `AppResolver` ánh xạ host với `host_pattern` của Application. Log không thể xử lý được bỏ qua có kiểm soát để consumer không kẹt ở một poison message.

`DetectionPipeline` tải rule đang bật qua Redis; pattern được biên dịch trong JVM. `RuleEngineImpl` chọn haystack theo `PATH|QUERY|UA|RAW`, tạo hit, rồi `RiskScorer` tính điểm. `AlertService` đảm bảo tối đa một Alert cho mỗi WebEvent. Tất cả thay đổi cốt lõi nằm trong transaction; Kafka/SSE fan-out chỉ chạy *after commit*, tránh phát thông báo cho dữ liệu đã rollback.

Độ tin cậy, chuẩn hóa và cache Tính idempotent dựa trên `raw_ref` và partial unique index, không dựa trên hash nội dung. Cách này phù hợp Kafka redelivery cùng offset, nhưng cùng một dòng được producer gửi lại ở offset mới vẫn thành event mới. Lỗi "unprocessable log" bị listener ghi warning rồi bỏ qua để offset tiếp tục, tránh poison message làm dừng partition. Đánh đổi là log hỏng bị mất vì chưa có dead-letter topic.

Parser runtime chỉ kiểm tra/trích field, xử lý envelope, chuẩn hóa host và giá trị rỗng; không URL-decode lặp, lowercase payload hay canonicalize whitespace. Ngược lại, rule-generator offline có `url_decode_loop()` và `canonical()`. Khoảng cách này giải thích tại sao một regex tốt trên corpus normalized vẫn có thể miss chuỗi encode khi chạy

thật. Việc Nginx ghi raw request path giúp giữ một phần bằng chứng nhưng không thay thế normalization thống nhất.

Application và danh sách rule enabled có TTL Redis 10 phút. Metadata của compiled rule có TTL 24 giờ; regex Java được tạo trong JVM. Khi Redis lỗi, cache Application và rule quay về DB. Host regex chưa có compiled cache; `host_pattern` được tạo lại khi resolve. Admin update rule chủ động invalidate hai cache, tránh chờ TTL.

4.3 Chính sách Alert và Incident

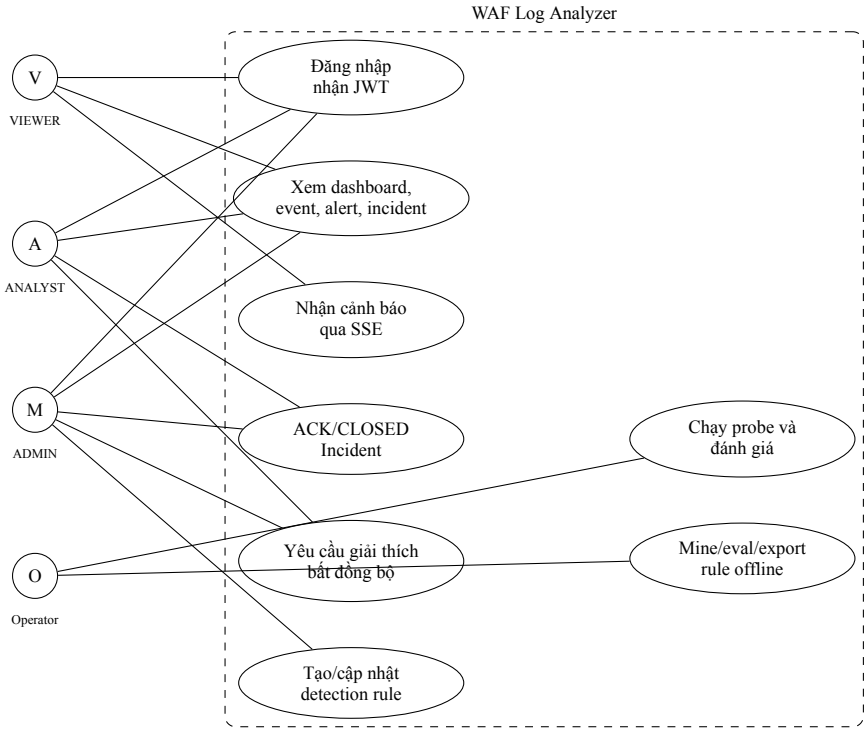
Bảng 8. Quy tắc promote Incident hiện thực trong hệ thống.

Nhánh	Điều kiện	Hành động
Không promote	Severity LOW	Giữ event/hit; không tạo Incident
Attach	Có Incident OPEN cùng $(appId, clientIp)$ trong 5 phút	Gắn Alert, cập nhật max score/severity và bộ đếm
IMMEDIATE	CRITICAL, tương ứng score $\geq$ 85	Tạo Incident ngay
AGGREGATE	MEDIUM/HIGH, chưa có Incident; ít nhất $K = 3$ Alert chưa gắn trong $W = 5$ phút	Tạo Incident và gắn cả batch
Stale close	Incident OPEN cũ hơn cửa sổ	Tự đóng, ghi SYSTEM note, sau đó đánh giá lại

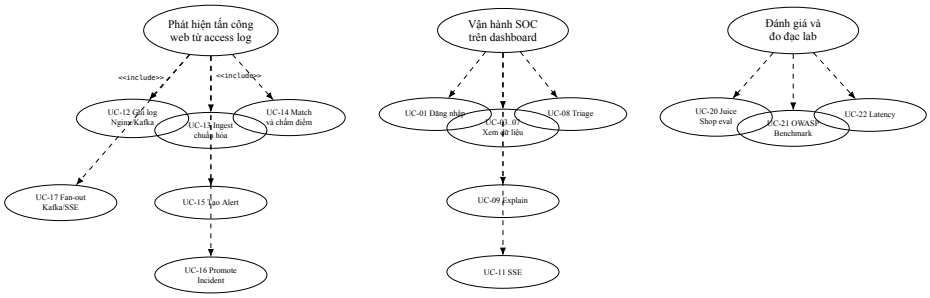
4.4 Use Case

Ba vai trò SOC kế thừa theo quyền. VIEWER chỉ đọc và stream; ANALYST thêm triage và yêu cầu explain; ADMIN thêm ghi luật. Operator ngoại tuyến vận hành lab, chạy probe và xuất rule. Hình 3 không biểu diễn chức năng chưa có như refresh token, quản lý user hay chỉnh Application.

Phân rã Use Case Hình 4 phân rã ba chức năng gốc thành các Use Case con theo quan hệ *include*. Việc phân rã bám trực tiếp mã nguồn: pipeline tự động tách thành ghi log, ingest, match/score, tạo Alert và tương quan Incident; nhóm SOC tách thành đăng nhập, quan sát và xử lý sự cố; nhóm đánh giá tách thành Juice Shop, OWASP Benchmark và đo độ trễ.



Hình 3. Use Case tổng quan theo chức năng đã hiện thực. Ký hiệu trong actor chỉ là viết tắt vai trò.



Hình 4. Phân rã Use Case: ba chức năng gốc và các Use Case con (*include*).

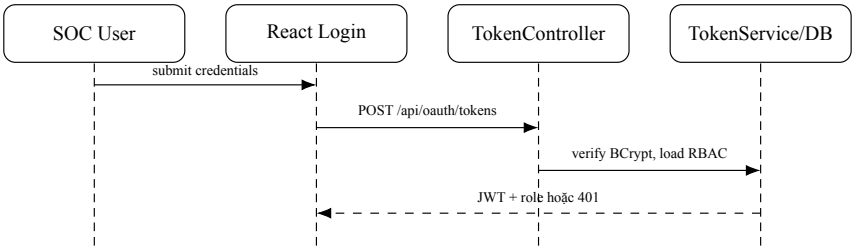
4.5 Danh mục và đặc tả đầy đủ các Use Case

Phần này chỉ liệt kê hành vi có đường thực thi trong mã nguồn. Các bảng hoặc permission chưa có controller như refresh-token rotation, `users:write`, `apps:write`, USER work note không được tính là Use Case đã hoàn thành. Use Case giao diện và API dùng chung service được gộp để tránh mô tả hai lần cùng một hành vi. Mỗi Use Case có tiền điều kiện, luồng chính, ngoại lệ, hậu điều kiện và sequence tương ứng.

Nhóm tương tác của người dùng SOC

UC-01 – Đăng nhập và cấp JWT.

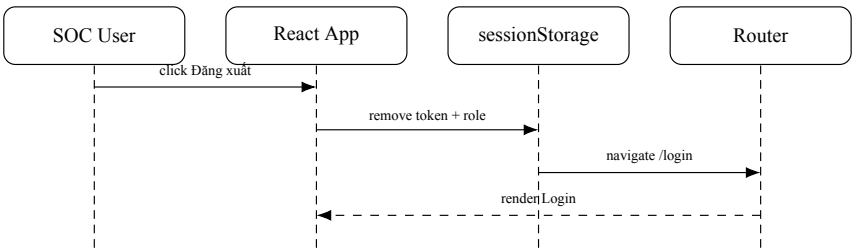
Tác nhân: người dùng SOC. **Tiền điều kiện:** backend hoạt động, tài khoản trong bảng `users` ở trạng thái enabled. **Luồng chính:** người dùng gửi username/password; frontend gọi `POST /api/oauth/tokens`; `TokenService` xác thực BCrypt, đọc role cùng permissions, ký JWT HS256 chứa `sub`, `role` và `permissions`. Frontend lưu access token và role trong `sessionStorage`. **Ngoại lệ:** thiếu trường trả 400; sai thông tin hoặc user bị disable trả 401. **Hậu điều kiện:** client có access token sống 480 phút; không có refresh token được phát hành.



Hình 5. Sequence UC-01: đăng nhập và cấp JWT.

UC-02 – Đăng xuất.

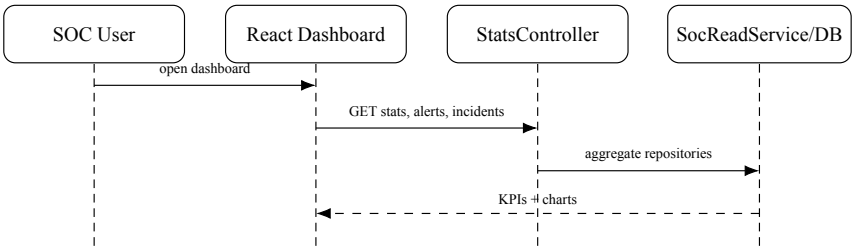
Tác nhân: người dùng đã đăng nhập. **Luồng chính:** `clearSession()` xóa token và role khỏi `sessionStorage`, sau đó router chuyển về `/login`. **Hậu điều kiện:** browser không còn gửi Bearer token. Đây là logout phía client; server không revoke JWT, do đó token bị sao chép vẫn hợp lệ tới khi hết hạn.



Hình 6. Sequence UC-02: đăng xuất phía client.

UC-03 – Xem dashboard tổng quan.

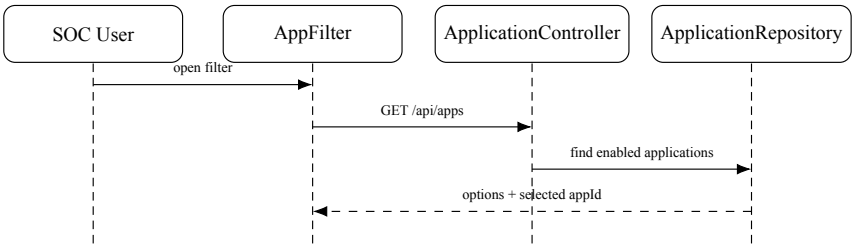
Tác nhân: VIEWER, ANALYST hoặc ADMIN. **Tiền điều kiện:** có quyền đọc stats, alerts, incidents và applications. **Luồng chính:** React gọi song song overview, alerts, incidents và applications; backend tổng hợp số event, hit, alert và incident; đồng thời dựng phân bố severity và timeline từ PostgreSQL. Dashboard đăng ký hai stream để reload KPI khi có notice mới. **Ngoại lệ:** 401 làm frontend xóa session; lỗi từng request được hiển thị như dữ liệu rỗng, không làm dừng pipeline.



Hình 7. Sequence UC-03: tải dashboard tổng quan.

UC-04 – Lọc theo Application.

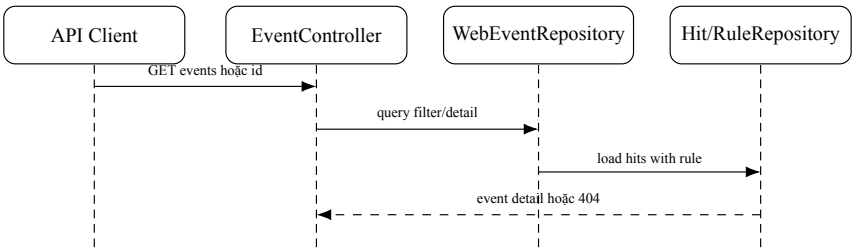
Tác nhân: mọi vai trò SOC với `apps:read`. **Luồng chính:** hook frontend gọi `/api/apps`; người dùng chọn app và `appId` được truyền tới các API hỗ trợ. Backend hiện chỉ có chức năng đọc Application. Hai bản ghi seed tách `juice.lab.local` và `shop.lab.local`; đây là minh họa scope, không phải hai workload độc lập.



Hình 8. Sequence UC-04: tải và áp dụng bộ lọc Application.

UC-05 – Tra cứu WebEvent và DetectionHit.

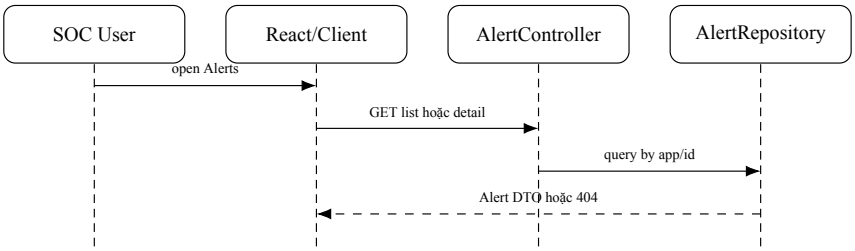
Tác nhân: API client có `events:read`; giao diện React chưa có trang tương ứng. **Luồng chính:** list hỗ trợ `appId`, `from`, `to`; detail tải Application và danh sách hit kèm rule. **Ngoại lệ:** id không tồn tại trả 404. **Hậu điều kiện:** không thay đổi dữ liệu; đây là đường truy vết từ request đã chuẩn hóa tới bằng chứng regex.



Hình 9. Sequence UC-05: tra cứu event và hit.

UC-06 – Xem danh sách và chi tiết Alert.

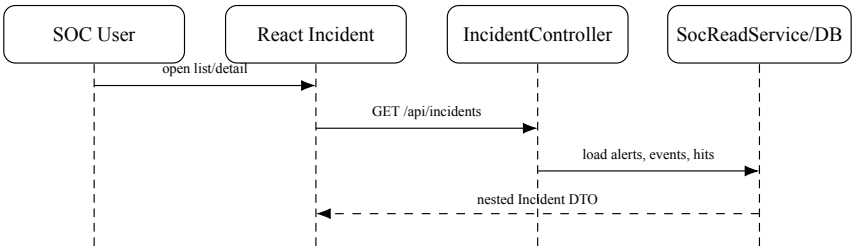
Tác nhân: mọi vai trò có `alerts:read`. **Luồng chính:** trang Alerts gọi `/api/alerts` với tham số `appId`, lọc thêm tại client và chèn notice SSE mới lên đầu. Endpoint detail tồn tại nhưng chưa được frontend dùng. Alert có thể chưa có Incident nếu không đủ điều kiện promote. **Hậu điều kiện:** read-only.



Hình 10. Sequence UC-06: duyệt Alert.

UC-07 – Xem danh sách và chi tiết Incident.

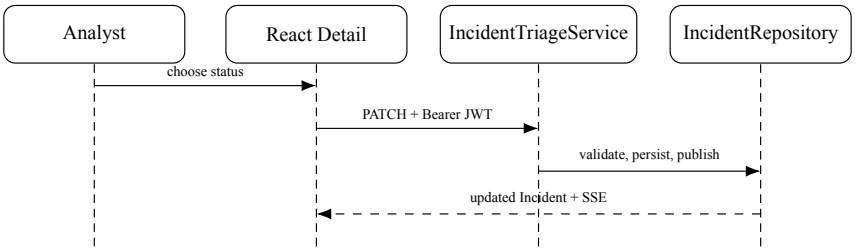
Tác nhân: mọi vai trò có `incidents:read`. **Luồng chính:** list nhận `appId`; giao diện lọc status, promote reason và explanation status. Detail dựng cây Incident–Alert–WebEvent–DetectionHit để analyst thấy toàn bộ căn cứ. **Ngoại lệ:** id sai trả 404. Hệ thống không trả USER work note vì chưa có API cho chức năng này.



Hình 11. Sequence UC-07: duyệt Incident và bằng chứng.

UC-08 – Triage Incident.

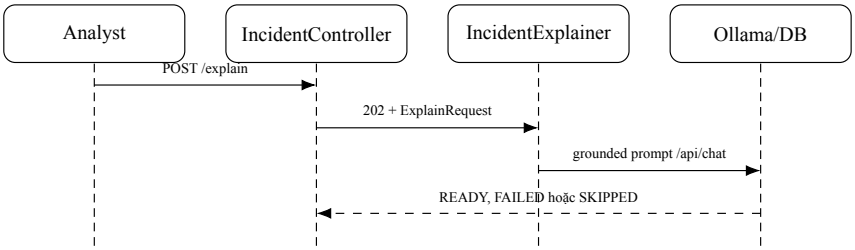
Tác nhân: ANALYST hoặc ADMIN có `incidents:triage`. **Tiền điều kiện:** Incident tồn tại và chưa CLOSED. **Luồng chính:** PATCH status `ACK` hoặc `CLOSED`; service lấy username từ JWT, ghi `triaged_by`, `acked_at`/`closed_at` và phát notice sau commit. **Ngoại lệ:** status khác hai giá trị trên trả 400; user không tồn tại trả 401; Incident CLOSED trả 409. VIEWER có thể nhìn thấy nút trong UI hiện tại nhưng API trả 403.



Hình 12. Sequence UC-08: ACK hoặc CLOSED Incident.

UC-09 – Yêu cầu giải thích Incident.

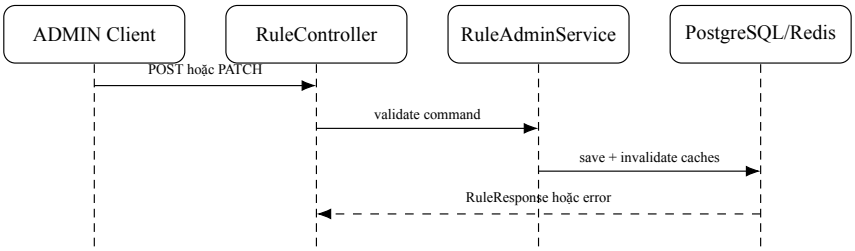
Tác nhân: ANALYST/ADMIN có `incidents:explain`. **Luồng chính:** API trả 202; khi LLM bật, trạng thái chuyển `PENDING`, sự kiện `ExplainRequest` được xử lý bất đồng bộ sau commit; worker dựng prompt từ tối đa năm Alert/hit, gọi Ollama, rồi ghi `READY` hoặc `FAILED`. Khi LLM tắt, trạng thái `SKIPPED`. **Ràng buộc:** model không được sửa score, category, Alert hoặc Incident status.



Hình 13. Sequence UC-09: giải thích Incident bất đồng bộ.

UC-10 – Quản trị DetectionRule.

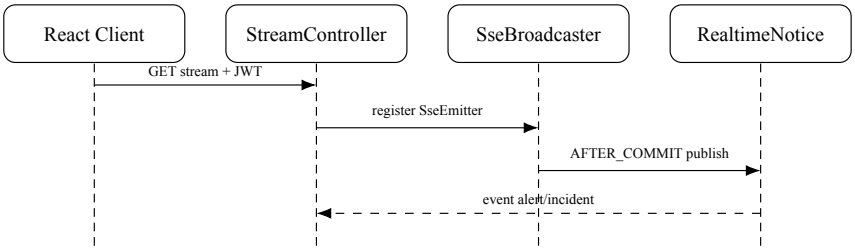
Tác nhân: đọc rule cho cả ba vai trò; tạo/cập nhật chỉ ADMIN với `rules:write`. **Luồng chính:** GET lọc theo enabled; POST tạo rule; PATCH thay category, target, pattern, weight, enabled hoặc version. Service kiểm tra trùng `ruleId`, lưu PostgreSQL, sau đó invalidate danh sách rule Redis và compiled-pattern key. **Ngoại lệ:** payload không hợp lệ trả 400, id thiếu trả 404, code trùng trả 409. Chức năng này API-only, chưa có màn hình React.



Hình 14. Sequence UC-10: tạo hoặc cập nhật rule.

UC-11 – Nhận cập nhật thời gian thực.

Tác nhân: browser có `stream:read`. **Luồng chính:** fetch-based SSE mở hai route `/api/stream/alerts` và `/api/stream/incidents`. Broadcaster giữ emitter trong một danh sách thread-safe, gửi event tên `alert`/`incident` và loại client lỗi. Frontend reconnect sau 2 giây. **Giới hạn:** emitter nằm trong memory nên không chia sẻ giữa nhiều backend instance; không có replay cursor khi mất kết nối.

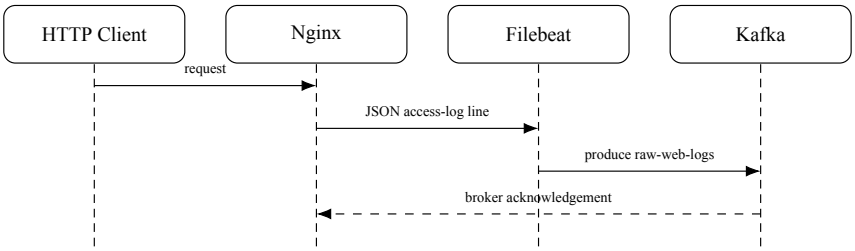


Hình 15. Sequence UC-11: đăng ký và nhận SSE.

Nhóm Use Case tự động của pipeline

UC-12 – Ghi và vận chuyển access log.

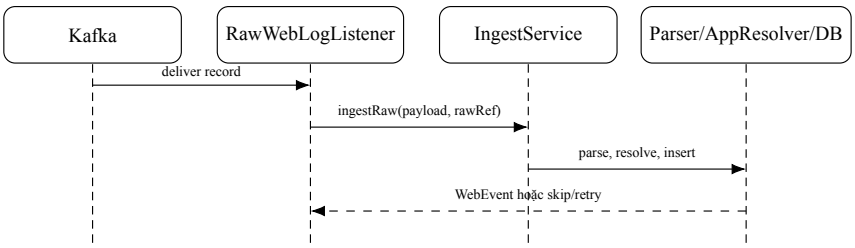
Tác nhân kích hoạt: HTTP request qua Nginx. **Luồng chính:** Nginx proxy request tới Juice Shop và ghi JSON vào file access log. Biến path tùy chỉnh lấy từ raw request URI để giữ dấu hiệu traversal. Filebeat filestream quét mỗi 1 giây, đóng gói log và produce topic `raw-web-logs` với gzip, `acks=1`. **Ngoại lệ:** không có DLQ ở tầng này.



Hình 16. Sequence UC-12: ghi log và đưa vào Kafka.

UC-13 – Consume, parse, resolve app và chống trùng.

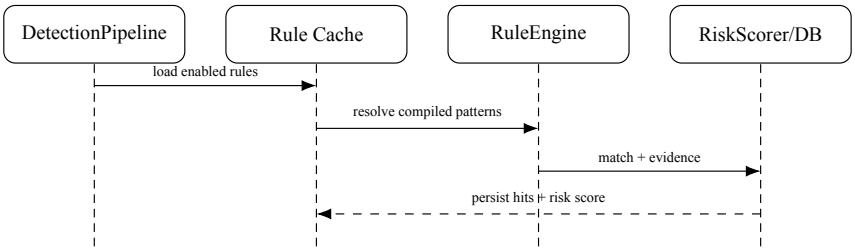
Tác nhân kích hoạt: Kafka record. Listener tạo `rawRef` từ topic–partition–offset. Parser nhận JSON Nginx thuần hoặc Filebeat envelope; host được trim, lowercase và bỏ port trước khi match regex `host_pattern`. Nếu `rawRef` đã tồn tại, service trả event cũ và không detect lại. Race insert được chặn bởi unique index rồi re-fetch. Log hỏng/host lạ bị ghi warning và bỏ qua; lỗi khác được rethrow cho Kafka container. Không có dead-letter topic.



Hình 17. Sequence UC-13: ingest và chuẩn hóa log.

UC-14 – Match rule, lưu hit và chấm điểm.

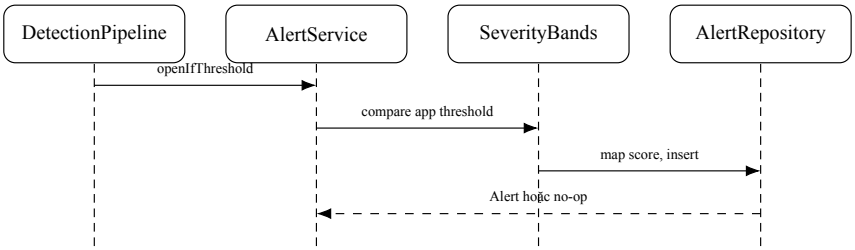
Luồng chính: lấy rule enabled từ Redis hoặc DB; chọn haystack `PATH|QUERY|UA|RAW`; `matcher.find()` tạo tối đa một hit cho mỗi rule, evidence bị cắt ở 500 ký tự. Regex sai bị skip có warning. Điểm base là tổng weight chặn ở 100; frequency bonus bằng $\min(20, 2(n-1))$ khi $n \geq 2$; status bonus 5 cho 403/404/500 nếu base dương. **Điểm cần lưu ý:** frequency query theo IP trong 5 phút, chưa scope theo Application.



Hình 18. Sequence UC-14: phát hiện và chấm điểm.

UC-15 – Mở Alert theo ngưỡng.

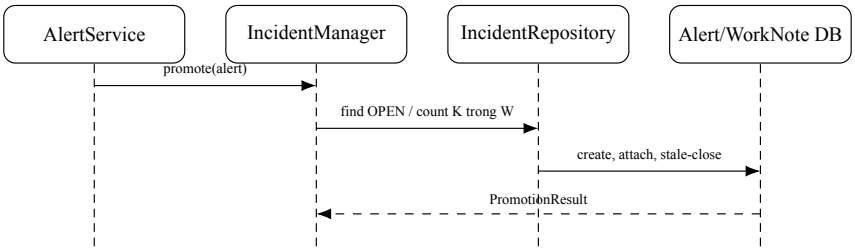
Tiền điều kiện: WebEvent đã có score. **Luồng chính:** nếu score nhỏ hơn threshold của Application thì kết thúc; ngược lại tạo một Alert với severity từ các band LOW < 40, MEDIUM ≥ 40, HIGH ≥ 70, CRITICAL ≥ 85. Unique `event_id` đảm bảo tối đa một Alert mỗi event. **Ngoại lệ:** duplicate trả Alert hiện có.



Hình 19. Sequence UC-15: tạo Alert.

UC-16 – Tương quan và promote Incident.

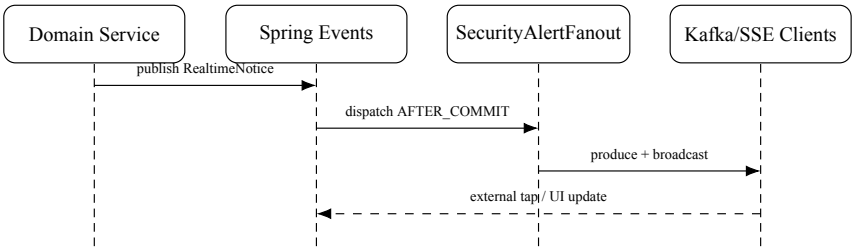
Luồng chính: LOW không promote; nếu có Incident OPEN cùng app+IP và chưa stale thì attach; CRITICAL tạo IMMEDIATE; MEDIUM/HIGH tạo AGGREGATE khi đủ ba Alert chưa gắn trong 5 phút. Incident stale bị CLOSED và ghi SYSTEM note. Partial unique index chỉ cho một Incident OPEN mỗi app+IP. **Rủi ro đồng thời:** code không dùng `SELECT FOR UPDATE` và không xử lý riêng unique violation khi hai request cùng tạo Incident, nên index là lớp bảo vệ cuối chứ chưa phải cơ chế retry hoàn chỉnh.



Hình 20. Sequence UC-16: promote hoặc attach Incident.

UC-17 – Fan-out sau commit.

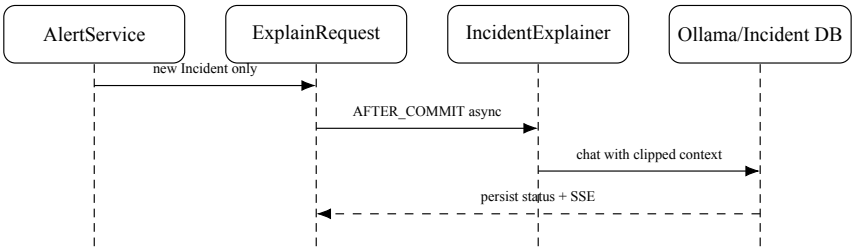
Tác nhân kích hoạt: `RealtimeNotice`. **Luồng chính:** `SecurityAlertFanout` chạy `@Async` và `AFTER_COMMIT`; serialize notice sang topic `security-alerts`, đồng thời gọi `SseBroadcaster`. Không có consumer topic này trong repository; React nhận SSE trực tiếp từ cùng Spring process. Việc chờ commit ngăn UI nhận entity đã rollback.



Hình 21. Sequence UC-17: phát notice tới Kafka và SSE.

UC-18 – Tự động giải thích Incident mới.

Khi promotion thực sự tạo Incident và LLM bật, `AlertService` phát `ExplainRequest`. Alert attach vào Incident cũ không kích hoạt lại model. Prompt loại control character, giới hạn từng field, dừng khoảng 5.000 ký tự và dùng tối đa năm Alert. Timeout cấu hình thực tế là 90 giây. Response rỗng, lỗi HTTP hoặc lỗi parse đều chuyển FAILED và phát SSE; ingest không bị block.

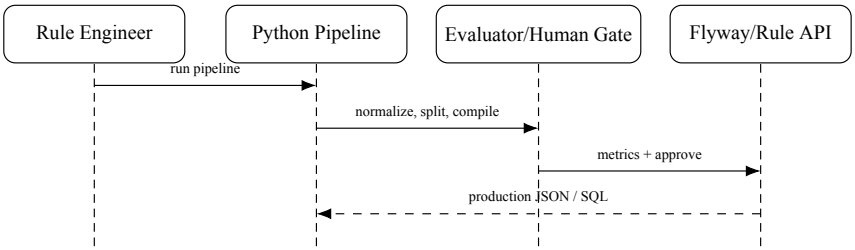


Hình 22. Sequence UC-18: auto-explain sau promote.

Nhóm Use Case vận hành và đánh giá ngoại tuyến

UC-19 – Sinh, kiểm định và xuất rule offline.

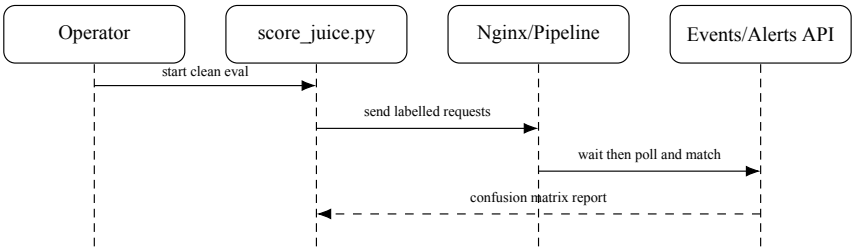
Tác nhân: rule engineer. Pipeline Python thực hiện download, ingest, normalize, dedupe, split, adversarial generation, compile, evaluate, promote và export Flyway. AI chỉ đề xuất abstract rule; compiler tạo regex Java-compatible; held-out bị ẩn khỏi bước sinh; human gate quyết định promotion. Runtime chỉ nhận artifact qua migration hoặc API, không gọi Python/LLM trong luồng Kafka.



Hình 23. Sequence UC-19: vòng đời rule offline.

UC-20 – Chạy probe và chấm Juice lab.

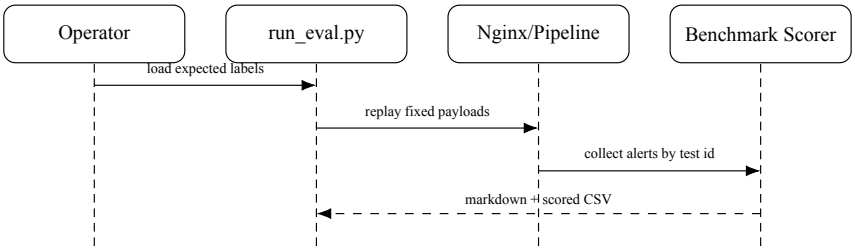
Tác nhân: operator. Script chờ login thành công, ghi nhận event id trước test, gửi 87 request trong ground truth, chờ mặc định 15 giây, poll events/alerts và ghép theo host/path/query gần đúng. Alert tồn tại là positive; hit dưới threshold vẫn tính FN. Kết quả ghi `juice-lab.md`. Ghép heuristic có thể nhầm khi traffic đồng thời.



Hình 24. Sequence UC-20: đánh giá Juice Shop.

UC-21 – Đánh giá external bằng OWASP Benchmark.

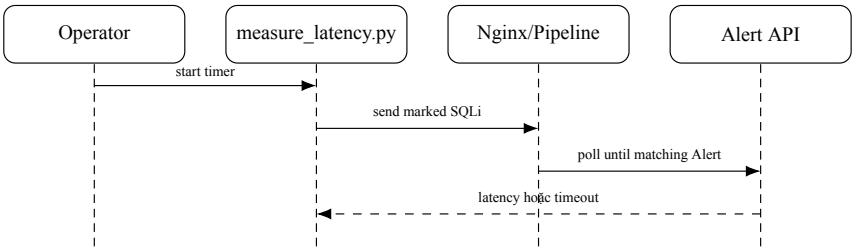
Runner đọc expected results và Java source upstream. Vector được chia thành bốn nhóm: thấy trong log, header/cookie, không xác định và ngoài phạm vi. Script gửi fixed payload cho 780 case nhìn thấy trong log rồi chấm theo Alert. Upstream không commit vào repository và phải tải theo README. Chính sách cấm sửa production rule sau khi xem kết quả.



Hình 25. Sequence UC-21: replay OWASP Benchmark.

UC-22 – Đo request–Alert latency.

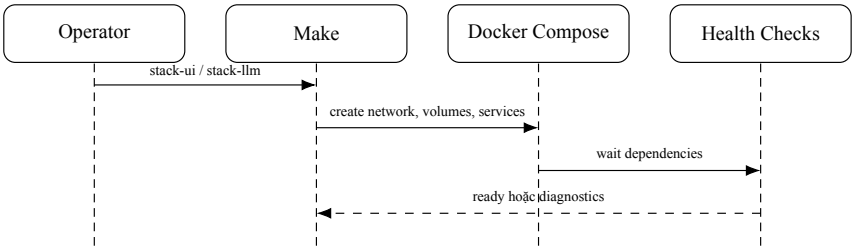
Script gắn marker vào một SQLi request, ghi timestamp ngay trước HTTP, sau đó poll Alert API cho tới timeout. Chênh lệch là request-to-persisted Alert quan sát qua API, không phải latency riêng của regex và không phải SSE p95. Artifact hiện tại tổng hợp nhiều lần chạy 11,5–15,1 giây.



Hình 26. Sequence UC-22: đo độ trễ đầu cuối.

UC-23 – Quản lý vòng đời lab.

Các target Make/Compose khởi động hạ tầng, backend, UI, LLM tùy chọn, chạy smoke/test và reset volume. `make reset` là thao tác phá hủy dữ liệu lab có chủ đích trước external eval. Một sai khác cần công bố: `application.yaml`/README mặc định LLM off nhưng Compose dùng `APP_LLM_ENABLED=true` nếu biến môi trường không được đặt; do đó operator phải đặt rõ biến để bảo đảm trạng thái SKIPPED.



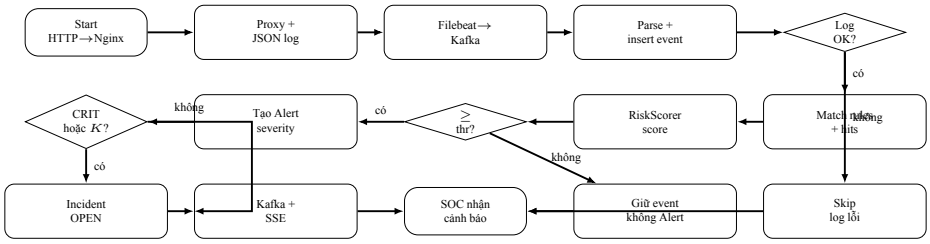
Hình 27. Sequence UC-23: khởi động và dừng lab.

Bảng 9. Chức năng có dấu vết dữ liệu nhưng chưa phải Use Case hoàn chỉnh.

Artifact/quyền	Hiện trạng	Không được tuyên bố
`refresh_tokens`	Table + entity, không repository/service/API	Đã có refresh rotation/logout server
`apps:write`	Permission seed; controller chỉ GET	ADMIN quản trị Application
`users:read/write`	Permission seed, không UserController	Đã có quản trị tài khoản
`incident_work_notes`	SYSTEM note khi stale; không API USER	Analyst ghi/chỉnh work note
`security-alerts`	Backend chỉ produce	React consume Kafka

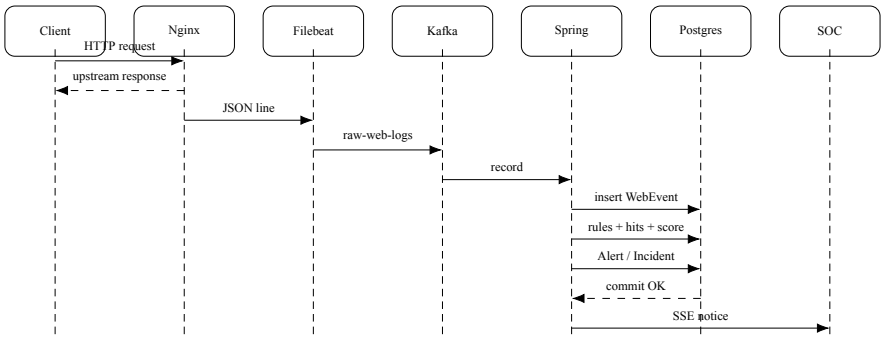
4.6 Phân tích thiết kế luồng chính

Luồng chính: *request* → *log* → *phát hiện* → *Alert/Incident* → *SOC*. Ba sơ đồ dưới đây là hoạt động, tuần tự và ERD của cùng luồng.



Hình 28. Sơ đồ hoạt động của luồng phát hiện chính.

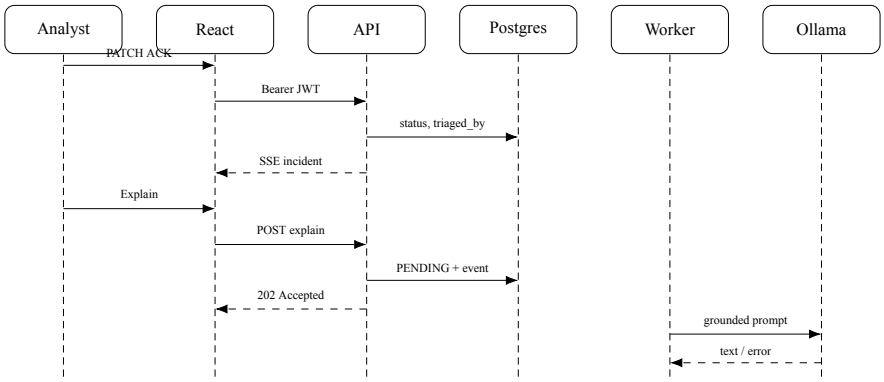
Sơ đồ hoạt động Hình 28: log lỗi bị skip; dưới threshold không tạo Alert; CRITICAL promote ngay; MEDIUM/HIGH cần đủ $K = 3$ trong 5 phút.



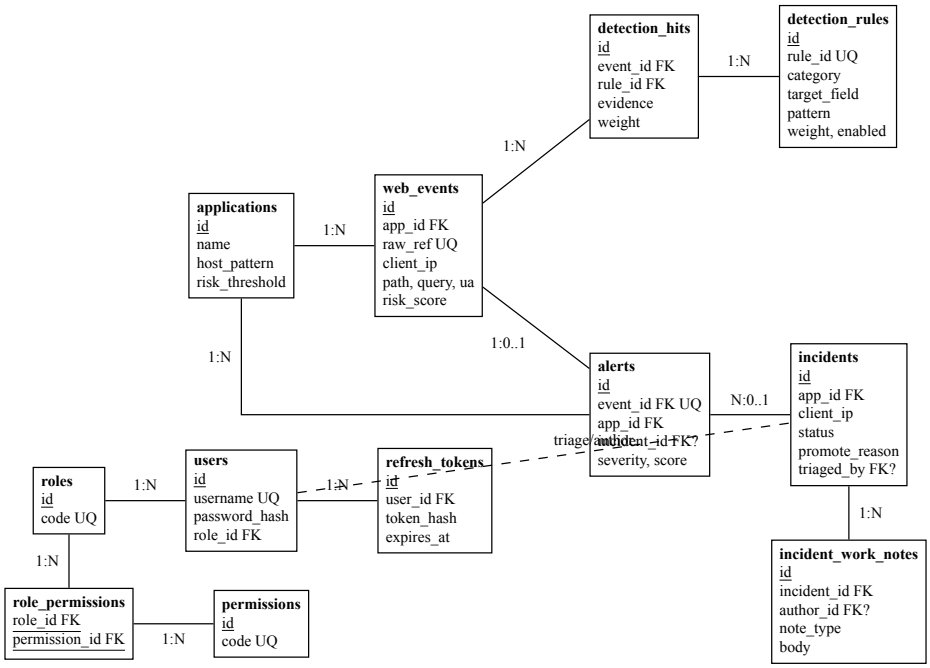
Hình 29. Sequence request–log–detect–alert–SSE.

Sơ đồ tuần tự Hình 29 nhấn mạnh hai mốc: persist trước fan-out, và Incident chỉ xuất hiện khi ngưỡng/chính sách được thỏa. Event dưới ngưỡng vẫn lưu WebEvent/hit nhưng không có Alert.

Sequence phụ: triage và giải thích Endpoint explain của Incident trả HTTP 202. Worker chạy bất đồng bộ sau commit, lấy tối đa năm Alert cùng các hit, cắt độ dài field, gọi Ollama rồi lưu trạng thái `READY` hoặc `FAILED`. Khi LLM tắt, trạng thái là `SKIPPED`; detector và triage vẫn hoạt động.



Hình 30. Sequence triage và giải thích Incident bất đồng bộ.



Hình 31. ERD rút gọn từ migration V1–V3.

Thiết kế cơ sở dữ liệu (ERD) Flyway là nguồn sự thật khi chạy Docker (`hibernate.ddl-auto=none`). Hình 31 phục vụ đúng luồng chính: một request thành `web_events`, có thể sinh nhiều `detection_hits`, tối đa một `alerts` theo `event_id`, rồi gắn vào `incidents`. `incident_id` nullable cho Alert chưa promote; tối đa một Incident OPEN trên khóa

$(app_id, client_ip)$. Bảng refresh token có schema nhưng chưa có service/API hoàn chỉnh.

5 Triển khai

5.1 Công nghệ và cấu hình

Máy chủ lab và toolchain host Các số liệu dưới đây lấy từ máy chạy lab khi biên soạn báo cáo và từ pin phiên bản trong `docker-compose.yml`, Dockerfile, `build.gradle`, `package.json`. Môi trường mục tiêu là lab cục bộ, không phải cluster production.

Bảng 10. Hệ điều hành host và công cụ phát triển/vận hành.

Thành phần	Phiên bản quan sát	Vai trò
Host OS	macOS 26.6.2 (Darwin 25.6.0, arm64)	Máy chạy Docker Desktop và script đánh giá
Docker Engine	27.4.0	Runtime container
Docker Compose	v2.31.0-desktop.2	Orchestration stack `waf-log-analyzer`
Node.js / npm	22.13.1 / 10.9.2	Build frontend (image dùng `node:22-alpine`)
Python 3	3.9.6	Runner Juice/Benchmark/latency (`datasets/eval/*.py`)
Java trong image	Eclipse Temurin 25 JDK/JRE	Build và chạy Spring Boot backend

Stack dịch vụ và thư viện Port lab tiêu biểu: Nginx `:80`, backend `:8080`, dashboard `:3001`, Postgres `:5432`, Redis `:6379`, Kafka host `:29092`, Kafka UI `:8088`. API áp dụng Spring Security resource server, JWT HS256 mang `role` và danh sách `permissions`. Password seed dùng BCrypt. Các route được bảo vệ bằng `hasAuthority` tại `SecurityConfig`; CSRF tắt vì API stateless, CORS giới hạn origin lab. Secret và tài khoản mặc định chỉ phù hợp lab, phải thay trong môi trường thực.

5.2 Quản trị và chính sách an ninh

Giải pháp được ánh xạ với NIST CSF 2.0 [15]: *Govern* định nghĩa owner, RACI, retention và ngưỡng; *Identify* quản lý Application và tài sản dữ liệu; *Protect* dùng RBAC, bí mật và cô lập mạng; *Detect* bao gồm log, rule, score; *Respond* gồm Incident, ACK/CLOSED, evidence và truyền thông SSE; *Recover* hiện mới ở mức reset/redeploy lab. Với ISO/IEC 27001:2022 [16], các kiểm soát phù hợp nhất là A.8.15 Logging, A.8.16 Monitoring, A.5.24–A.5.28 về quản lý sự cố và thu thập bằng chứng.

6 Kiểm thử và đánh giá

6.1 Thiết kế thí nghiệm và metric

Các tầng được giữ độc lập để tránh suy diễn sai:

Bảng 11. Môi trường triển khai thực tế trong repository.

Thành phần	Phiên bản/công nghệ	Trách nhiệm
Edge/workload	Nginx 1.27-alpine; Juice Shop `bkimminich/juice-shop:v20.1.1`	Reverse proxy, JSON access log và ứng dụng mục tiêu
Streaming	Filebeat 8.17.4; Apache Kafka 4.1.2 (KRaft); Kafka UI `kafbat/kafka-ui:v1.2.0`	Ship log; topic `raw-web-logs`/`security-alerts`
Backend runtime	Java 25; Spring Boot 4.1.1; Spring Dependency Management 1.1.7; Spring Cloud BOM 2025.1.3	Ingest, rule engine, score, Alert/Incident, REST/SSE, security
Backend libs	Spring Security/OAuth2 Resource Server; Spring Data JPA; Spring Kafka; Spring Data Redis; Flyway + `flyway-database-postgresql`; SpringDoc OpenAPI 3.1.0; Spring AI 2.0.1 (Ollama); Resilience4j; Lombok; PostgreSQL JDBC	Persistence, messaging, API docs, explain tùy chọn
Persistence	PostgreSQL 16-alpine; Redis 7.4-alpine	System of record, migration Flyway, cache Application/rule
Frontend	React 19.1; React DOM 19.1; React Router 7.6.2; Recharts 2.15.3; Vite 6.3.5; `@vitejs/plugin-react` 4.5.2; phục vụ bằng Nginx 1.27-alpine	Dashboard, danh sách, drill-down, triage và realtime
AI tùy chọn	Ollama 0.32.6; model mặc định `qwen3.5:9b`; Spring AI Ollama starter	Explain Incident bất đồng bộ; không tham gia phát hiện
Đóng gói	Docker Compose; network bridge `waf-net`; profile `ui`/`llm`	Lab cô lập; port quản trị bind `127.0.0.1`

Bảng 12. Ma trận quyền chính đã hiện thực.

Chức năng	VIEWER	ANALYST	ADMIN
Xem event/alert/incident/stats/app	✓	✓	✓
Nhận SSE	✓	✓	✓
ACK/CLOSED Incident	–	✓	✓
Yêu cầu Incident explain	–	✓	✓
Đọc rule	✓	✓	✓
Tạo/cập nhật rule	–	–	✓

Bảng 13. Chính sách vận hành đề xuất cho lab và trách nhiệm RACI.

Hoạt động	Chính sách	RACI
Thu thập	Traffic thử nghiệm phải đi qua Nginx; không ghi password, token hoặc body nhạy cảm	Operator (R), owner hệ thống (A)
Retention	Giữ event/alert/incident 30 ngày trong lab; xóa volume khi kết thúc học phần	Operator (R), owner (A)
Rule change	Rule mới phải có provenance, version, held-out/FPR gate và review trước khi enable	Rule engineer (R), owner (A), analyst (C)
Triage	CRITICAL xử lý ngay; Incident khác ACK trong phiên demo; CLOSED chỉ khi đã ghi nhận kết luận	Analyst (R), owner (A)
Evidence	Giữ raw reference, event, hit và timestamp; không sửa kết quả đánh giá sau khi xem Benchmark	Analyst (R), giảng viên (I)
Sự cố lab	Cô lập container, lưu log cần thiết, reset credential/volume, sau đó khôi phục Compose	Operator (R), owner (A)

– **Tầng A – offline:** pattern chạy trên held-out, benign và adversarial payload; chưa có ngưỡng runtime hay độ trễ pipeline.
– **Tầng B – Juice Shop:** gửi GET đã gắn nhãn qua Nginx, đợi ingest và chấm theo việc Alert có tồn tại; bao gồm score threshold.
– **External:** replay các case OWASP Benchmark nhìn thấy trong log để kiểm tra giao thức/nhãn ngoài; không retune rule.
– **Độ trễ:** đo từ thời điểm gửi request tới khi API quan sát Alert; chưa phải p95 SSE hoặc benchmark tải.

Với TP, FP, FN, TN:

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}.$$

6.2 Kịch bản vận hành, kiểm thử và đo đạc

Để đáp ứng yêu cầu trình bày ít nhất một kịch bản vận hành/kiểm thử có đo đạc, báo cáo chọn thí nghiệm Juice Shop đầu–cuối làm kịch bản đại diện. Đây là luồng thật trên stack Docker, không phải unit test fixture.

1. **Chuẩn bị lab:** Operator chạy `make reset` (hoặc stack sạch tương đương), bật profile UI, xác nhận Nginx `127.0.0.1:80` và dashboard `:3001` healthy.
2. **Gửi traffic gắn nhãn:** Script `datasets/eval/score_juice.py` gửi 87 request GET qua Nginx: 50 browse sạch và 37 probe (SQLi/XSS/Path Traversal/MIXED). Mỗi probe có nhãn kỳ vọng Alert hoặc clean đã khóa trước khi chạy.
3. **Chờ pipeline:** Script đợi ingest Filebeat→Kafka→Spring, rồi gọi API Event/Alert để ghép theo query/path.
4. **Quan sát vận hành SOC:** Analyst đăng nhập dashboard, thấy Alert/Incident realtime (SSE), mở sự cố và có thể ACK/CLOSED.

5. **Đo đạc:** Xuất confusion matrix (Bảng 15) và có thể chạy thêm `measure_latency.py` để đo request–Alert.

Kết quả đo được của kịch bản này: precision 1,000; recall 0,774 trên 31 probe cần cảnh báo; 0 FP trên 56 request sạch; latency request–Alert quan sát khoảng 11,5–15,1 giây. Các tầng khác (offline ruleset, OWASP Benchmark, latency chuyên biệt) được báo cáo riêng ở các mục tiếp theo để không trộn mẫu số.

6.3 Kết quả luật offline

Mười bốn rule qua cổng precision $\geq 0{,}80$, FPR $\leq 0{,}02$. Trên 12.099 benign parameter cho mỗi rule, FPR báo cáo bằng 0; điều này không đồng nghĩa không có FP trong production. Coverage là hợp của cả ruleset theo category, không phải tổng recall từng rule.

Bảng 14. Coverage ruleset tầng A.

Category	Held-out n	Coverage	Adversarial n	Coverage	Benign FPR
SQLI	6.598	50,52%	32.851	49,58%	0
XSS	229	68,12%	1.111	69,85%	0
PATH_TRAVERSAL	175	48,00%	808	51,36%	0

6.4 Mô phỏng tấn công trên Juice Shop

Ground truth có 87 request: 50 browse sạch và 37 probe, trong đó 31 request được kỳ vọng có Alert, sáu probe được gắn nhãn clean. Đây là simulation có nhãn bằng `curl`, không phải pentest đầy đủ bằng ZAP, Nuclei hay sqlmap. Kết quả ở Bảng 15 cho thấy precision 1,000 và recall 0,774 trên toàn tập; browse-only không có mẫu positive nên precision/recall không xác định, không phải bằng 0 về năng lực.

Bảng 15. Confusion matrix theo slice của thí nghiệm Juice Shop.

Slice	TP	FP	FN	TN	Precision	Recall	FPR	$TPR - FPR$
Browse	0	0	0	50	–	–	0	–
Probe	24	0	7	6	1,000	0,774	0	0,774
Tất cả	24	0	7	56	1,000	0,774	0	0,774

Theo category của các probe positive (loại MIXED), recall SQLI là 5/11 = 0,455; XSS 9/9 = 1,000; Path Traversal 10/10 = 1,000. Bảy FN gồm:

– `sqli-time-sleep`, `sqli-time-pgsleep`, `sqli-time-benchmark`;

– `sqli-string-union`, `sqli-stacked-sleep`, `sqli-comment-union`; và `hard-multi`.

Một số mẫu có hit nhưng tổng điểm 20–42 thấp hơn 60; một số không khớp pattern. Đây là bằng chứng để điều chỉnh weight/rule trong vòng đời sau, không được đổi nhãn hậu nghiệm.

Độ bất định thống kê Vì số mẫu nhỏ, điểm ước lượng cần đi cùng khoảng tin cậy. Khoảng Wilson 95% cho recall tổng $24/31$ xấp xỉ [0,602; 0,886]; SQLi $5/11$ là [0,213; 0,720]; XSS $9/9$ là [0,701; 1,000]; Path Traversal $10/10$ là [0,722; 1,000]. Do đó kết quả 100% của XSS/traversal không chứng minh recall production bằng 100%; nó chỉ cho thấy chưa quan sát FN trong mẫu rất nhỏ.

Tương tự, quan sát 0 FP không có nghĩa FPR thật bằng 0. Theo "rule of three", cận trên 95% xấp xỉ $3/n$: với 56 clean request của Juice, cận trên khoảng 5,36%; với 12.099 benign value tầng A, khoảng 0,025%. Phép tính giả định mẫu Bernoulli độc lập, trong khi payload có thể cùng nguồn/cụm, nên chỉ dùng để diễn giải mức bất định chứ không phải chứng nhận chất lượng.

6.5 OWASP Benchmark v1.2

OWASP Benchmark là một ứng dụng Java chứa các test case có chủ đích, đi kèm file expected results cho biết vị trí nào thực sự dễ tổn thương [14]. Mục tiêu chính thức là đo khả năng một công cụ bảo mật phân biệt *đoạn mã vulnerable* và *đoạn mã safe*, chứ không chỉ nhận ra chuỗi payload nguy hiểm. Bản v1.2 có 2.740 case thuộc 11 nhóm lỗ hổng; ba category liên quan đề tài gồm SQL injection, XSS và Path Traversal.

Cách OWASP Benchmark tính điểm Với mỗi test case, nhãn ground truth Y lấy từ `expectedresults-1.2.csv`; verdict $\hat{Y}$ là việc công cụ có báo case đó là vulnerable hay không:

Bảng 16. Ý nghĩa confusion matrix theo giao thức OWASP Benchmark.

Ô	Điều kiện	Ý nghĩa chính thức
TP	$Y = true, \hat{Y} = alert$	Công cụ báo đúng case có lỗ hổng
FN	$Y = true, \hat{Y} = no\ alert$	Bỏ sót case có lỗ hổng
FP	$Y = false, \hat{Y} = alert$	Báo case an toàn là có lỗ hổng
TN	$Y = false, \hat{Y} = no\ alert$	Không báo đúng case an toàn

Benchmark dùng:

$$TPR = \frac{TP}{TP + FN}, \qquad FPR = \frac{FP}{FP + TN}, \qquad Score = TPR - FPR.$$

Score nằm trong $[-1, 1]$. Giá trị 1 là lý tưởng ($TPR = 1, FPR = 0$); giá trị 0 biểu thị verdict không phân biệt được hai nhóm tại threshold đang xét. Một classifier luôn

báo positive và một classifier có TPR bằng FPR đều có thể nhận score 0; vì vậy score 0 không đồng nghĩa "phát hiện được 0% payload".

Giao thức local được hiện thực Runner trong repository là một *external adaptation*, không phải lần chạy OWASP Benchmark chuẩn:

1. Đọc expected results và Java source của từng case.
2. Chỉ giữ ba category `sqli`, `xss`, `pathtraver`.
3. Phân vector theo API đọc input trong source. Query/parameter là log-visible; cookie, header, body và vector chưa rõ được tách riêng.
4. Lấy đường dẫn từ annotation servlet.
5. Gửi GET qua Nginx/Juice Shop với tên case làm parameter. Runner không deploy hay thực thi BenchmarkJava WAR.
6. Dùng một fixed payload cho mỗi family, được chọn từ họ probe sẵn có để đủ ngưỡng Alert 60:
   - SQLi: boolean tautology kết hợp `UNION SELECT`;
   - XSS: tag script kết hợp `javascript:`;
   - Traversal: dot-dot kết hợp encoded traversal.
7. Chờ ingest 20 giây, lấy toàn bộ Event/Alert, tìm `BenchmarkTest{id}` trong path/query và đánh dấu `got_alert`.

Payload không được lấy từ Benchmark để sinh hoặc retune production rule. Tuy nhiên, cùng một payload tấn công được gửi vào cả case nhãn `true` và `false`. Vì target thực sự vẫn là Juice Shop chứ không phải servlet Benchmark, nhãn "vulnerable/safe" không mô tả code đang xử lý request trong lần replay này.

Bảng 17. Phân rã toàn bộ 2.740 case OWASP Benchmark v1.2.

Bucket	Số case	Tỷ lệ	Cách xử lý
Log-visible query/parameter	780	28,47%	Replay và đưa vào TP/FP/FN/TN
Header/cookie invisible	329	12,01%	Đếm riêng; schema log không có
Unknown vector	118	4,31%	Không chấm vì parser source không xác định chắc input vector
Ngoài ba category	1.513	55,22%	Không tính FN
Tổng	**2.740**	**100%**	

Phạm vi mẫu được chấm Như vậy 1.960/2.740 case, tương đương 71,53%, không nằm trong confusion matrix. Việc loại này đúng với scope access-log và ba category, nhưng làm kết quả không đại diện điểm OWASP Benchmark toàn bộ. Giá trị `body_invisible` bằng 0 trong lần phân loại hiện tại không có nghĩa hệ thống thấy POST body; nó chỉ có nghĩa không case body nào của ba category đi qua nhánh đó trong corpus/source parser hiện dùng.

Bảng 18. Kết quả external eval trên 780 case log-visible.

Category	TP	FP	FN	TN	TPR	FPR	Score
SQLI	194	119	0	0	1,000	1,000	0
XSS	190	131	0	0	1,000	1,000	0
PATH_TRAVERSAL	83	63	0	0	1,000	1,000	0
Tổng	**467**	**313**	**0**	**0**	**1,000**	**1,000**	**0**

Bằng chứng thực nghiệm từ lần chạy sạch Ngày 11/10/2026, nhóm thực hiện `make reset` để xóa volume và dựng lại toàn bộ lab, tắt LLM bằng `APP_LLM_ENABLED=false`, sau đó chạy `run_eval.py` đúng một lần sau khi Nginx sẵn sàng. Hình 32 ghi lại tiến trình gửi đủ 780/780 request: mọi lần kiểm tra theo lô đều nhận HTTP 200, runner chờ 20 giây cho pipeline ingest rồi xuất Markdown và CSV. Dòng cuối là confusion matrix thực đo, không phải số liệu giả lập hoặc điền thủ công.

```
$ python3 datasets/eval/owasp-benchmark/run_eval.py
=== replaying 780 log-visible Benchmark GETs ===
… 50/780 last_http=200
… 100/780 last_http=200
… 150/780 last_http=200
… 200/780 last_http=200
… 250/780 last_http=200
… 300/780 last_http=200
… 350/780 last_http=200
… 400/780 last_http=200
… 450/780 last_http=200
… 500/780 last_http=200
… 550/780 last_http=200
… 600/780 last_http=200
… 650/780 last_http=200
… 700/780 last_http=200
… 750/780 last_http=200
… 780/780 last_http=200
waiting 20s for ingest…
wrote /Users/huylq/Desktop/IT6027-26.1A01-G21-WAF-log-analyzer/datasets/eval/owasp-benchmark/owa
TP=467 FP=313 FN=0 TN=0 TPR=1.0000 FPR=1.0000 score=0.0000
excluded body=0 header_cookie=329 oos=1513
```

Hình 32. Kết quả terminal của lần chạy OWASP Benchmark adaptation trên lab sạch: 467 TP, 313 FP, 0 FN, 0 TN và score 0.

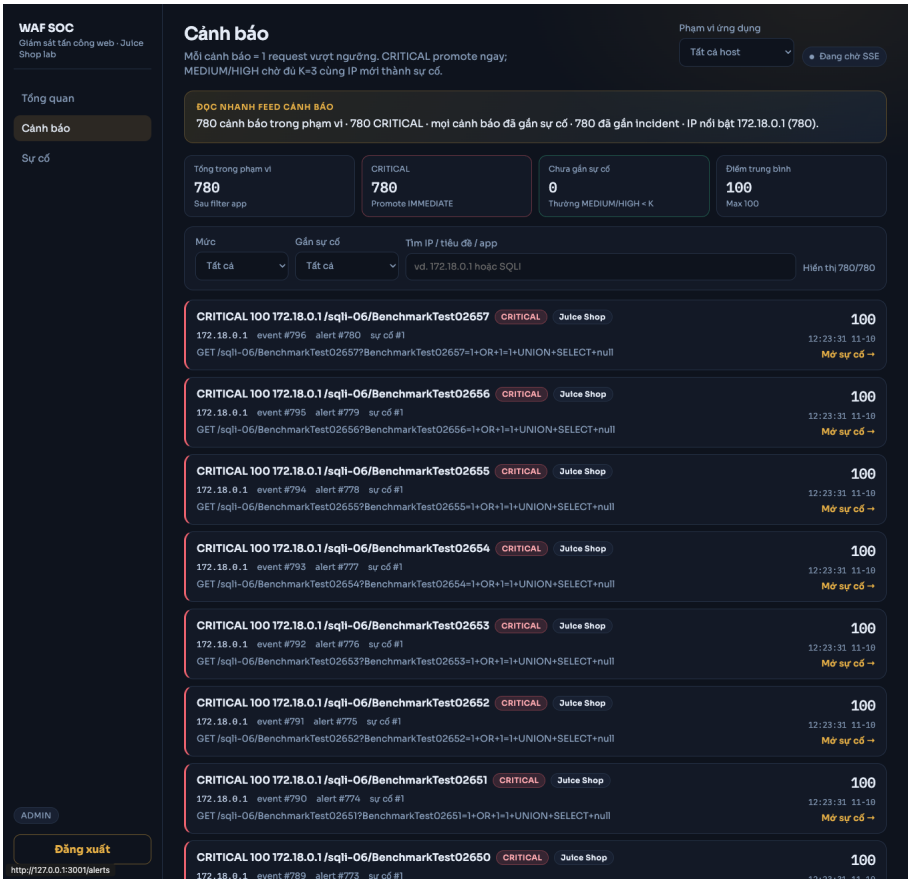
Ảnh dashboard ở Hình 33 là phép kiểm tra chéo độc lập với file kết quả của runner. Giao diện đọc dữ liệu thật từ REST API cho thấy 801 WebEvent (780 request Benchmark và 21 request khởi động/health-check), đúng 780 Alert, 780 Alert mức CRITICAL và một Incident OPEN do cơ chế `IMMEDIATE` gộp cùng ứng dụng/IP. Vì vậy, con số một

Incident không có nghĩa chỉ phát hiện một request; quan hệ 780 Alert–một Incident là kết quả của correlation key $(appId, clientIp)$ trong cửa sổ thời gian.



Hình 33. Dashboard SOC sau lần replay: 801 event được ingest, 780 Alert, 780 CRITICAL và một Incident được tương quan.

Hình 34 phóng to lớp bằng chứng Alert. Màn hình hiển thị 780/780 bản ghi, điểm trung bình 100 và các test id `BenchmarkTest0265x` còn nguyên trong path/query, cho phép truy vết từ case đầu vào đến WebEvent, Alert và Incident. Các ảnh chỉ là bằng chứng quan sát; artifact có thể kiểm tra bằng máy vẫn là `owasp-benchmark.md` và `scored-cases.csv`.



Hình 34. Danh sách Alert thật sau replay, thể hiện test id, payload SQLi, event id, alert id, incident id, severity và risk score.

Diễn giải phép tính score bằng số cụ thể Trong 780 case được chấm có 467 nhãn `true` và 313 nhãn `false`. Production rules tạo Alert cho cả 780 request:

$$TPR = \frac{467}{467 + 0} = 1,000,$$
$$FPR = \frac{313}{313 + 0} = 1,000,$$
$$Score = 1,000 - 1,000 = 0,000.$$

Không có FN vì toàn bộ 467 case nhãn true đều nhận Alert. Không có TN vì toàn bộ 313 case nhãn false cũng nhận Alert. Kết quả theo category có cùng hình dạng: mọi request đều positive, nên từng category đều có $TPR = FPR = 1$ và score 0.

Nếu tính thêm metric không phải score chính thức, precision của verdict theo nhãn Benchmark là:

$$Precision = \frac{467}{467 + 313} = 0,5987,$$

accuracy cũng bằng $467/780 = 0,5987$, còn balanced accuracy là $(TPR+TNR)/2 = (1 + 0)/2 = 0,5$. Accuracy 59,87% chỉ phản ánh tỷ lệ case vulnerable trong subset khi classifier luôn báo positive; nó không cho thấy khả năng phân biệt.

Tại sao detector báo cả case `false`? Nhãn `false` của Benchmark không có nghĩa request benign. Nó có nghĩa implementation của test case xử lý input an toàn và không tạo lỗ hổng. Runner vẫn gửi chuỗi như `UNION SELECT`, tag script hoặc `../../`. Bộ phân tích access log được thiết kế để nhận ra *attack attempt*; nó không có static/data-flow knowledge để biết servlet đích parameterize SQL, encode output hay canonicalize path.

Do đó:

- Với SOC, cảnh báo một attack attempt nhắm vào endpoint đã vá vẫn có thể hữu ích cho threat hunting.
- Với OWASP Benchmark, cùng cảnh báo đó là FP vì câu hỏi đánh giá là "code có vulnerable không?".
- Score 0 ở đây chứng minh detector không phân loại *vulnerable-code state*; nó không phủ định việc detector nhận ra 780/780 request có payload tấn công.

Đây là lệch ontology giữa hai bài toán, không chỉ là threshold sai. Giảm threshold không tạo TN; tăng threshold đủ cao có thể biến mọi verdict thành negative, khi đó TPR và FPR cùng 0 và score vẫn 0. Muốn score dương, công cụ phải đưa ra tỷ lệ positive cao hơn trên case truly vulnerable so với case safe, điều mà signature chỉ nhìn request không đủ thông tin để thực hiện.

Giá trị và giới hạn của thí nghiệm Thí nghiệm vẫn có giá trị ở ba điểm: kiểm tra pipeline chịu được 780 request có định danh; xác nhận payload mạnh đi qua Nginx–Kafka–rule engine tới Alert; và minh họa rủi ro dùng sai ground truth. Tuy nhiên không được dùng kết quả để tuyên bố hệ thống "đạt" hoặc "trượt" OWASP Benchmark theo nghĩa chính thức, vì:

- BenchmarkJava WAR không được deploy; code mang nhãn không xử lý request replay.
- Payload là fixed family payload, không phải request/payload gốc riêng của từng Benchmark case.
- Chỉ 28,47% corpus được chấm và chỉ ba category.
- Các case `false` không nhận benign control; tất cả request đều chứa attack syntax.
- Scoring lấy Alert sau threshold 60, không đo riêng rule hit; Event–case được ghép bằng test name và có thể bị ảnh hưởng nếu lab không sạch.

Giao thức cải tiến đề xuất Có hai phép đánh giá khác nhau cần tách:

1. **Đánh giá chuẩn Benchmark:** deploy BenchmarkJava, phát đúng request từng test case, rồi dùng một công cụ có nhiệm vụ kết luận endpoint vulnerable/safe. Khi đó nhãn expected results và verdict cùng trả lời một câu hỏi.

2. **Đánh giá attack-attempt detector:** bỏ nhãn vulnerable-code; gắn mọi request chứa fixed attack payload là positive, đồng thời gửi benign control tương ứng tới từng route để đo FP. Có thể tạo 780 attack + 780 benign request, khóa trước payload, threshold và quy tắc ghép event. Giao thức này phù hợp sản phẩm hiện tại hơn nhưng không được gọi là official OWASP Benchmark score.

Kết quả external này vì vậy được báo cáo riêng, không gộp với coverage tầng A hoặc confusion matrix Juice. Juice trả lời "request gắn nhãn có tạo Alert không"; Tier A trả lời "regex có match payload không"; OWASP Benchmark chính thức trả lời "công cụ có phân biệt code vulnerable và safe không". Ba mẫu số và ba câu hỏi không thể cộng thành một accuracy duy nhất.

6.6 Độ trễ, kiểm thử mã và giá trị nội tại

Probe latency SQLi đo thời gian request–Alert API trong nhiều lần chạy là 11,5–15,1 giây, vượt mục tiêu dưới 5 giây. HTTP chỉ khoảng 0,02 giây; quan sát hiện có quy phần lớn trễ cho Filebeat–Kafka–consumer, nhưng chưa có tracing thành phần để khẳng định tỷ lệ chính xác. Vì chưa đo p50/p95 dưới tải, kết quả không đại diện benchmark throughput.

Backend có unit test cho parser, rule engine, scorer, AlertService, IncidentManager, fan-out và LLM explain. Các test này kiểm tra logic fixture, không tự động xác nhận artifact đánh giá Python. Giá trị nội tại bị đe dọa bởi probe do tác giả soạn, ghép event theo query gần đúng, một lần chạy lab và payload external cố định. Giá trị ngoại tại bị giới hạn bởi một workload Juice Shop, ba category và không có traffic production.

7 Thảo luận về quản trị rủi ro

Giải pháp làm tốt tính truy vết: từ Incident có thể quay về Alert, WebEvent, hit và rule version. Phân tách Alert–Incident giảm số case SOC so với quy tắc một cảnh báo là một sự cố. LLM bị cô lập khỏi verdict, giúp lỗi model không làm mất bằng chứng. Ngược lại, hệ thống hiện chưa có audit trail đầy đủ cho thay đổi rule/triage, không có rotation refresh token dù schema đã chuẩn bị, chưa có quản lý người dùng và chưa ký/mã hóa log bất biến. Đây là khoảng trống quản trị cần ưu tiên trước khi dùng ngoài lab.

Rủi ro riêng tư phát sinh vì query và user-agent có thể chứa PII hoặc token. Chính sách 30 ngày chỉ là đề xuất lab; production phải dựa trên mục đích xử lý, phân loại dữ liệu, yêu cầu pháp lý và khả năng xóa. Quyền đọc log nên theo least privilege, truy cập phải audit, và dashboard không được hiển thị secret nguyên dạng.

8 Kết luận và hướng phát triển

Đề tài đã hiện thực một chuỗi đầu cuối có thể chạy bằng Compose: thu log, stream, chuẩn hóa, phát hiện bằng 14 rule, chấm điểm, tạo Alert/Incident, hiển thị SOC và giải thích tùy chọn. Use Case, sequence và ERD đều được đối chiếu với controller, service và migration hiện tại. Kết quả Juice cho thấy khả năng phát hiện tốt với XSS/Path Traversal trong tập probe, nhưng SQLi và latency chưa đạt kỳ vọng. External Benchmark khẳng định cần chọn metric đúng với câu hỏi an ninh.

Các hướng phát triển ưu tiên là:

1. instrument timestamp tại Nginx, Filebeat, Kafka, consumer và SSE để tối ưu p95;
2. bổ sung telemetry body/header theo allow-list, masking và giới hạn kích thước;
3. xây dựng regression corpus và quy trình approval/rollback rule;
4. bổ sung audit log bất biến, refresh-token rotation, quản lý user/app và secret manager;
5. triển khai feedback false-positive có human approval;
6. tích hợp WAF inline ở chế độ DetectionOnly trước khi cân nhắc block; và đánh giá đa ứng dụng, nhiều lần chạy, dưới tải đồng thời.

Mọi tự động hóa block hoặc điều chỉnh rule phải giữ con người trong vòng quyết định.

Acknowledgments. Tác giả sử dụng các dự án mã nguồn mở và bộ dữ liệu được trích dẫn để xây dựng môi trường thực hành. Các probe chỉ được thực hiện trên OWASP Juice Shop tự lưu trữ.

Disclosure of Interests. Tác giả không có xung đột lợi ích liên quan đến nội dung báo cáo.

Tài liệu

1. OWASP Foundation: OWASP Top 10: The Ten Most Critical Web Application Security Risks (2021), https://owasp.org/Top10/, last accessed 2026/10/11
2. OWASP Foundation: SQL Injection, https://owasp.org/www-community/attacks/SQL_Injection, last accessed 2026/10/11
3. OWASP Foundation: Cross Site Scripting (XSS), https://owasp.org/www-community/attacks/xss/, last accessed 2026/10/11
4. OWASP Foundation: Path Traversal, https://owasp.org/www-community/attacks/Path_Traversal, last accessed 2026/10/11
5. Information Security Institute, CSIC: HTTP DATASET CSIC 2010, https://www.isi.csic.es/dataset/, last accessed 2026/10/11
6. Morzeux: HttpParamsDataset, https://github.com/Morzeux/HttpParamsDataset, last accessed 2026/10/11
7. OWASP Foundation: OWASP ModSecurity Core Rule Set, https://owasp.org/www-project-modsecurity-core-rule-set/, last accessed 2026/10/11
8. Wu, Y., et al.: WAFBooster: Automatic Boosting of Web Application Firewalls. arXiv preprint arXiv:2501.14008 (2025)
9. Babaey, M., Ravindran, B.: GenXSS: An AI-driven Framework for Automated XSS Payload Generation and WAF Rule Creation. arXiv preprint arXiv:2504.08176 (2025)
10. Scano, G., et al.: ModSec-Learn: Boosting ModSecurity with Machine Learning. arXiv preprint arXiv:2406.13547 (2024), https://arxiv.org/abs/2406.13547
11. Chen, Y., et al.: RuleCache: Accelerating Web Application Firewalls by On-line Learning Traffic Patterns. In: IEEE International Conference on Web Services (ICWS), pp. 222–231 (2022). https://doi.org/10.1109/ICWS55610.2022.00041
12. Kreps, J., Narkhede, N., Rao, J.: Kafka: A Distributed Messaging System for Log Processing. In: Proceedings of the NetDB, pp. 1–7 (2011)
13. OWASP Foundation: OWASP Juice Shop, https://owasp.org/www-project-juice-shop/, last accessed 2026/10/11
14. OWASP Foundation: OWASP Benchmark, https://owasp.org/www-project-benchmark/, last accessed 2026/10/11

15. Pascoe, C., Quinn, S., Scarfone, K.: The NIST Cybersecurity Framework (CSF) 2.0. NIST CSWP 29, National Institute of Standards and Technology (2024). https://doi.org/10.6028/NIST.CSWP.29
16. International Organization for Standardization: ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements. ISO, Geneva (2022)
17. VMware: Spring Security Reference, https://docs.spring.io/spring-security/reference/, last accessed 2026/10/11
18. Elastic: Filebeat Reference, https://www.elastic.co/guide/en/beats/filebeat/current/, last accessed 2026/10/11
19. F5, Inc.: NGINX Documentation, https://nginx.org/en/docs/, last accessed 2026/10/11
20. PostgreSQL Global Development Group: PostgreSQL 16 Documentation, https://www.postgresql.org/docs/16/, last accessed 2026/10/11